使用 Flutter 建構字謎遊戲

來源：https://codelabs.developers.google.com/codelabs/flutter-word-puzzle?hl=zh-tw

步驟數：10

瞭解如何建構會耗用大量運算資源的 Flutter 應用程式，並維持 Flutter 流暢的互動。

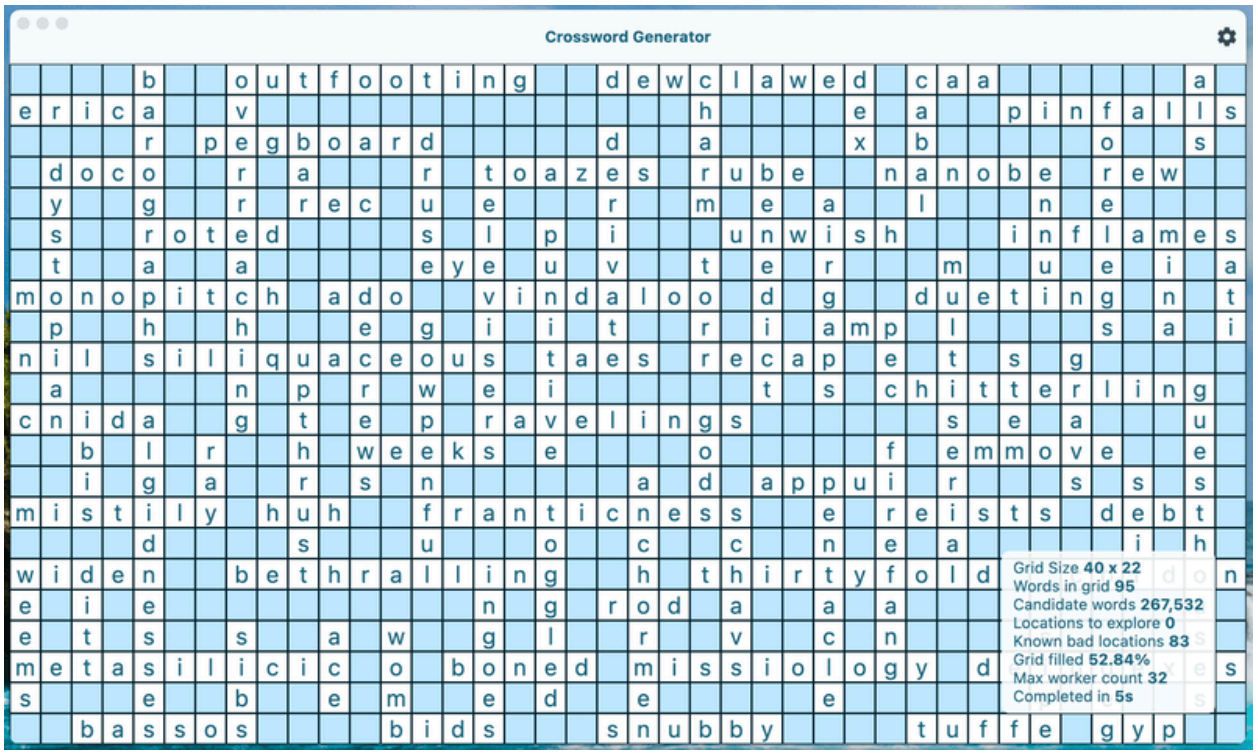
目錄

步驟 1 · 約 2 分鐘

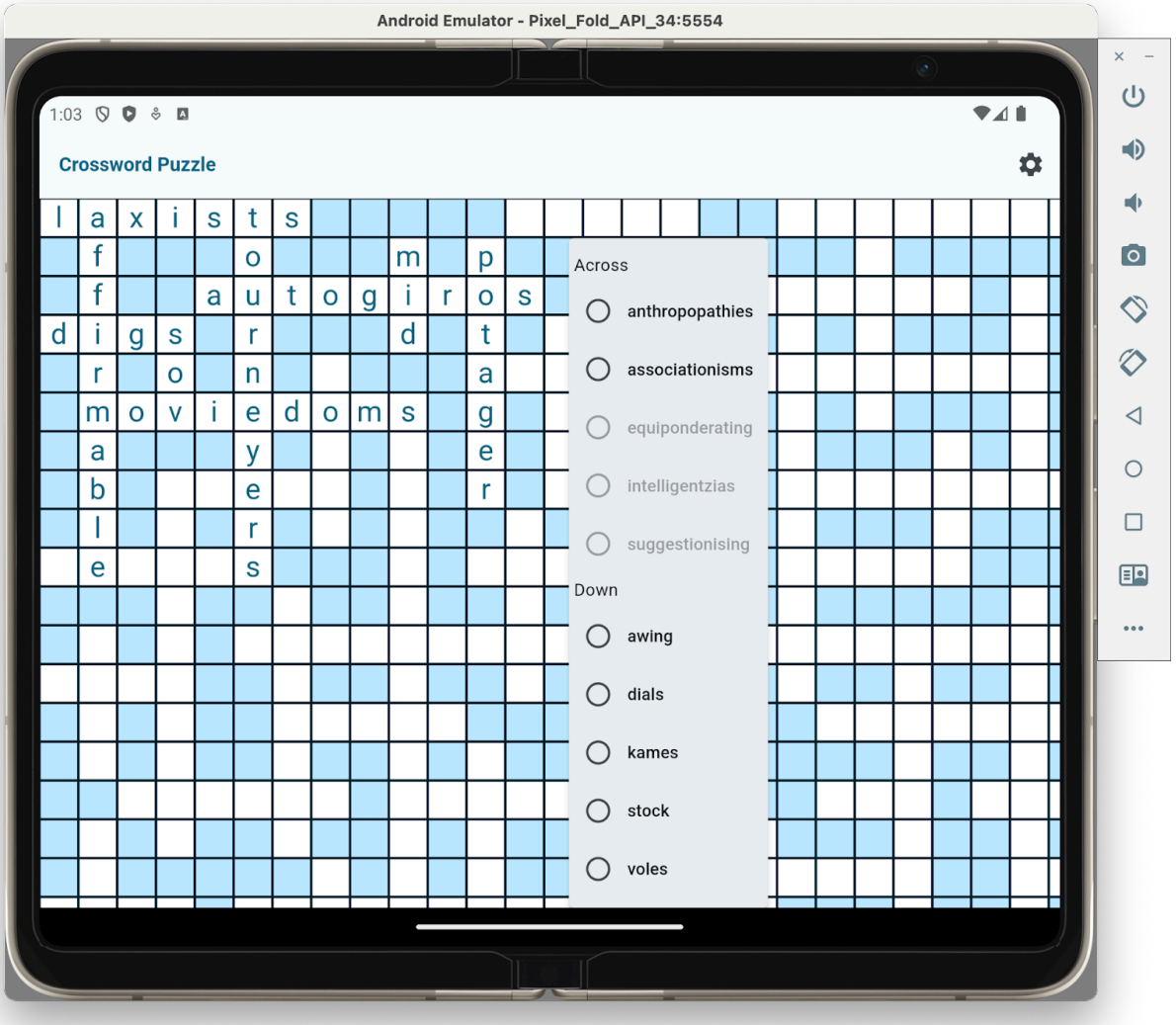
1. 事前準備

假設有人問你是否有可能製作出全世界最大的填字遊戲，您回想起在學校學過的一些 AI 技術，並想知道是否能使用 Flutter 探索演算法選項，為運算密集型問題建立解決方案。

在本程式碼研究室中，您將瞭解如何達成上述目標。最後，您將建構工具，在建構字謎的演算法空間中運作。有效填字遊戲的定義有很多種，這些技巧可協助您建構符合*您的*定義的填字遊戲。



以這項工具為基礎，然後使用填字遊戲產生器製作填字遊戲，供使用者解答。這款謎題適用於 Android、iOS、Windows、macOS 和 Linux。Android 裝置：



必要條件

- 完成「您的第一個 Flutter 應用程式」程式碼研究室

課程內容

- 如何使用隔離區執行耗用大量運算資源的工作，同時不以 Flutter 的 `compute` 函式和 Riverpod 的 `select` 重建篩選器值快取功能，阻礙 Flutter 的算繪迴圈。
- 如何運用 `built_value` 和 `built_collection` 實作以搜尋為基礎的傳統 AI (GOFAI) 技術，例如深度優先搜尋和回溯，進而善用不可變動的資料結構。
- 如何使用 `two_dimensional_scrollables` 套件的功能，以快速直覺的方式顯示格線資料。

摘要：本程式碼研究室會教導三項重要概念：

1. **效能**：使用隔離區，在大量運算期間維持 UI 回應速度
2. **資料管理**：運用不可變動的資料結構，打造高效率演算法
3. **UI 最佳化**：實作選擇性重建，確保格線順暢轉譯

需求條件

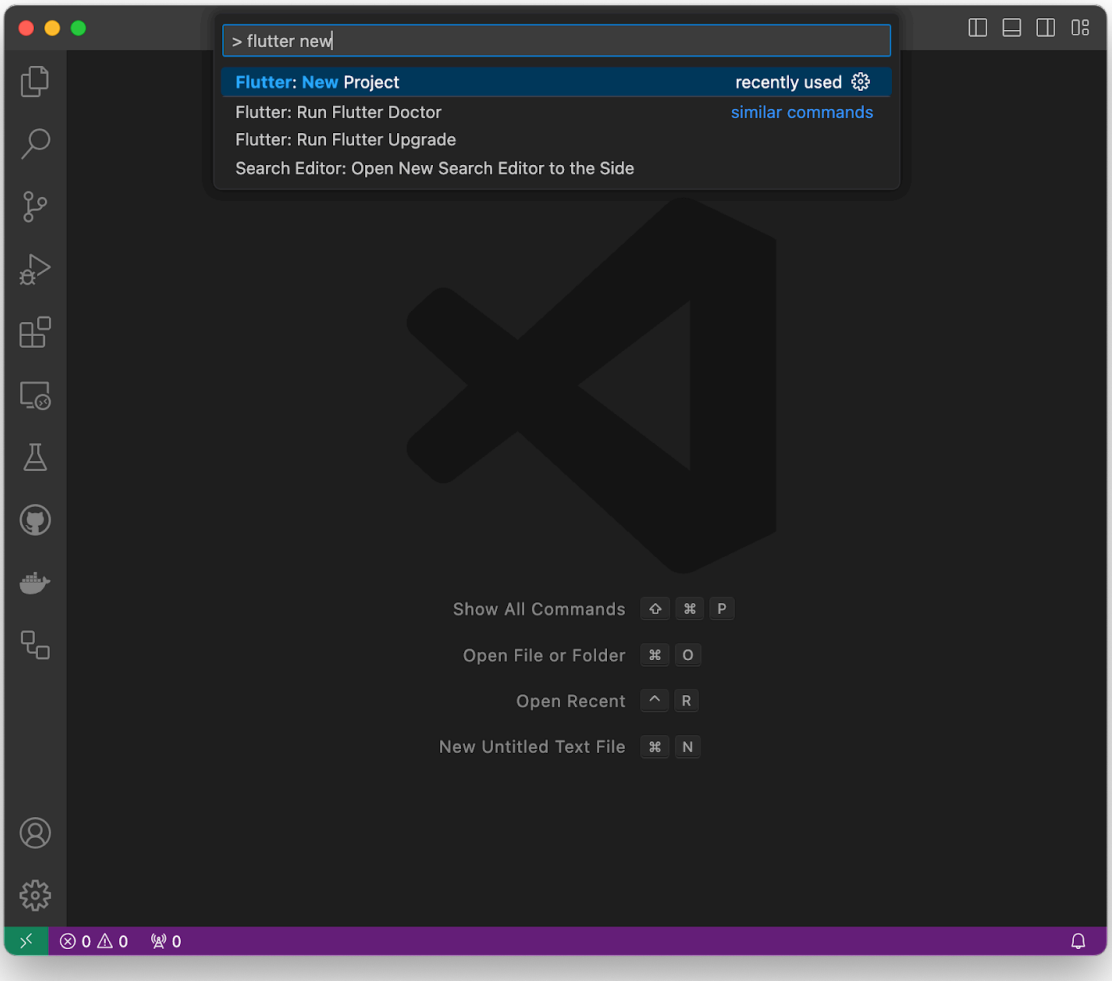
- Flutter SDK。
- Visual Studio Code (VS Code)，以及 Flutter 和 Dart 外掛程式。
- 所選開發目標的編譯器軟體。這個程式碼研究室適用於所有電腦平台、Android 和 iOS。如要以 Windows 為目標，您需要 VS Code；如要以 macOS 或 iOS 為目標，您需要 Xcode；如要以 Android 為目標，則需要 Android Studio。

步驟 2 · 約 5 分鐘

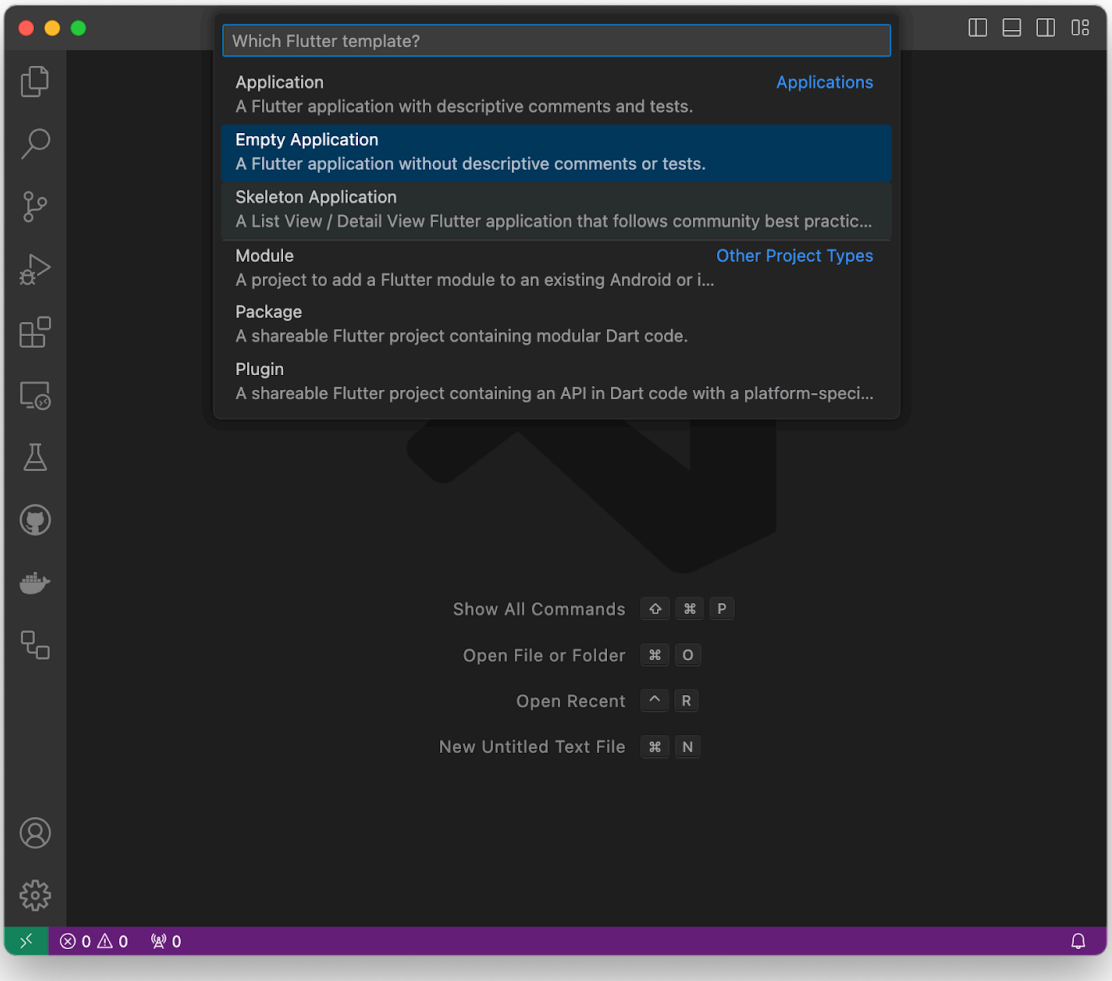
2. 建立專案

建立第一個 Flutter 專案

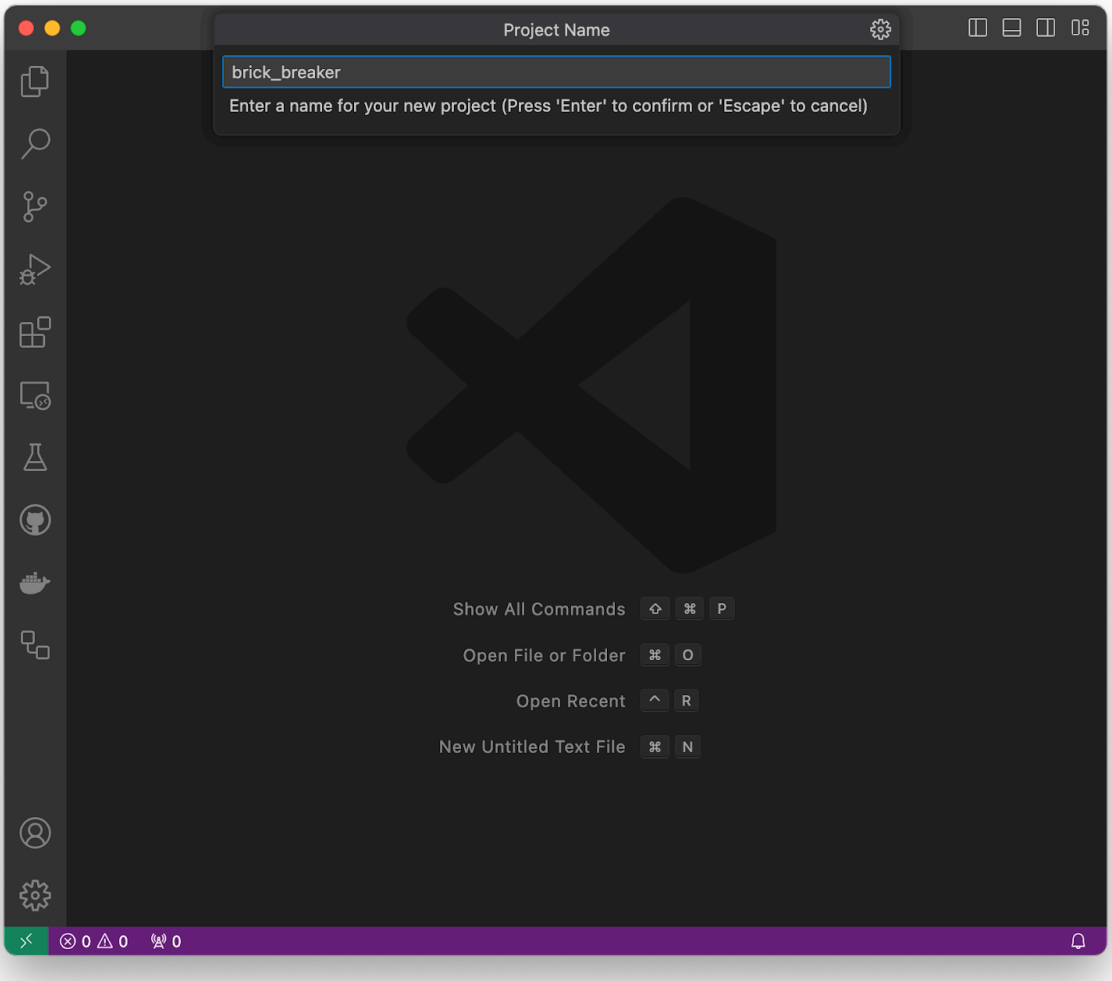
1. 啟動 VS Code。
2. 開啟指令面板 (Windows/Linux 為 Ctrl+Shift+P，macOS 為 Cmd+Shift+P)，輸入「flutter new」，然後在選單中選取「Flutter: New Project」。



3. **選取「Empty application」(空白應用程式)，然後選擇要建立專案的目錄。**這個目錄不得需要提升權限，路徑中也不得有空格。例如主目錄或 `C:\src\`。



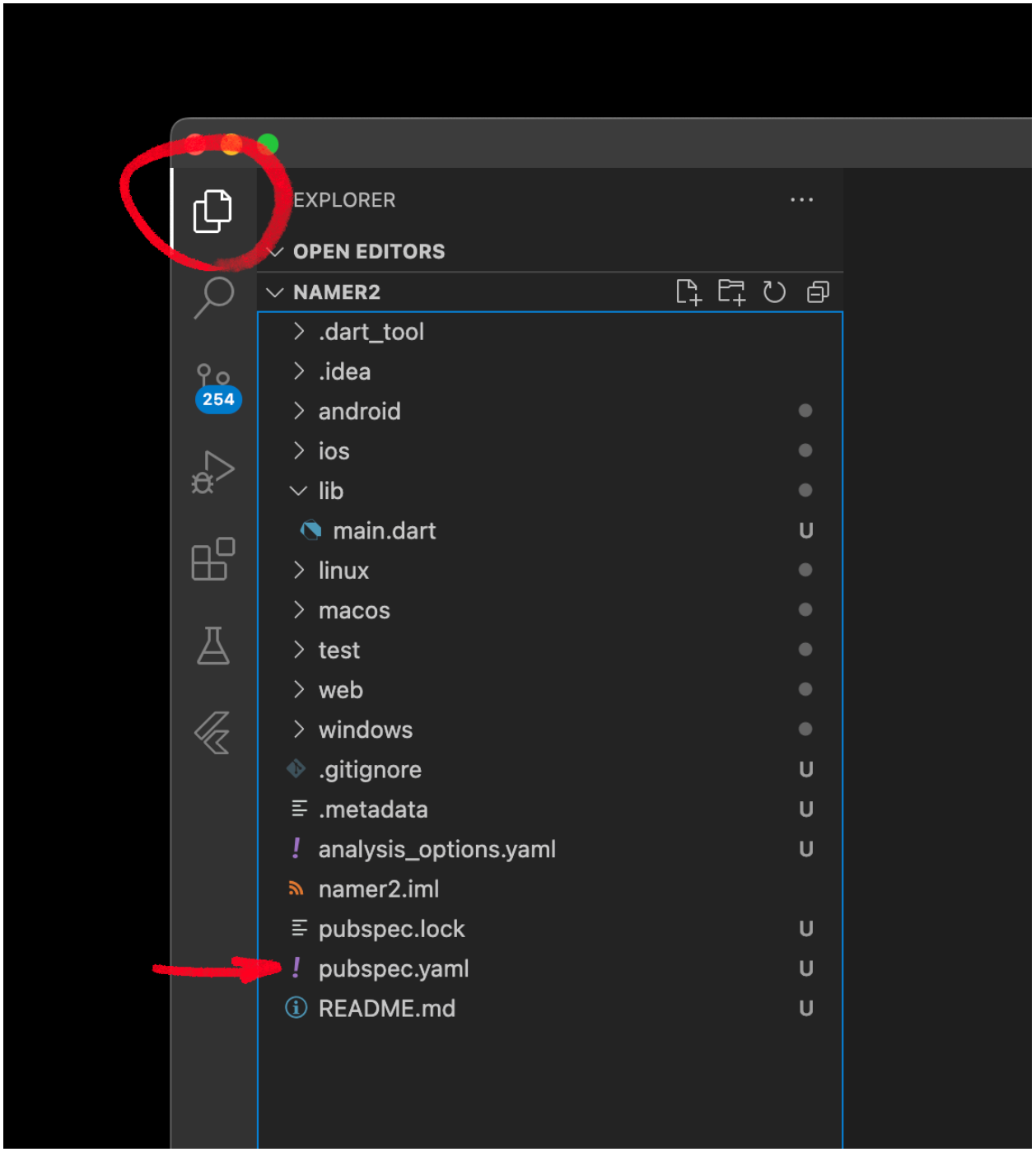
4. 為專案命名 `generate_crossword`。本程式碼研究室的其餘部分會假設您將應用程式命名為 `generate_crossword`。



Flutter 現在會建立專案資料夾，並在 VS Code 中開啟該資料夾。現在要以應用程式的基本架構覆寫兩個檔案的內容。

複製並貼上初始應用程式

1. 在 VS Code 的左側窗格中，按一下「Explorer」並開啟 `pubspec.yaml` 檔案。



2. 將這個檔案的內容替換為生成填字遊戲所需的下列依附元件：

pubspec.yaml

```
name: generate_crossword
description: "A new Flutter project."
publish_to: 'none'
version: 0.1.0

environment:
  sdk: ^3.9.0

dependencies:
  flutter:
    sdk: flutter
  built_collection: ^5.1.1
  built_value: ^8.10.1
  characters: ^1.4.0
  flutter_riverpod: ^2.6.1
  intl: ^0.20.2
  riverpod: ^2.6.1
  riverpod_annotation: ^2.6.1
  two_dimensional_scrollables: ^0.3.7

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^5.0.0
  build_runner: ^2.5.4
  built_value_generator: ^8.10.1
  custom_lint: ^0.7.6
  riverpod_generator: ^2.6.5
  riverpod_lint: ^2.6.5

flutter:
  uses-material-design: true
```

`pubspec.yaml` 檔案會指定應用程式的基本資訊，例如目前版本和依附元件。您會看到一系列依附元件，這些元件不屬於一般的空白 Flutter 應用程式。在接下來的步驟中，您會用到所有這些套件。

> **主要依附元件說明：**
>
> - `built_value/built_collection`：不可變動的資料結構，可提升演算法效率
> - `flutter_riverpod`：狀態管理與效能最佳化
> - `two_dimensional_scrollables`：高效能格狀顯示
> - `characters`：正確處理 Unicode 字串

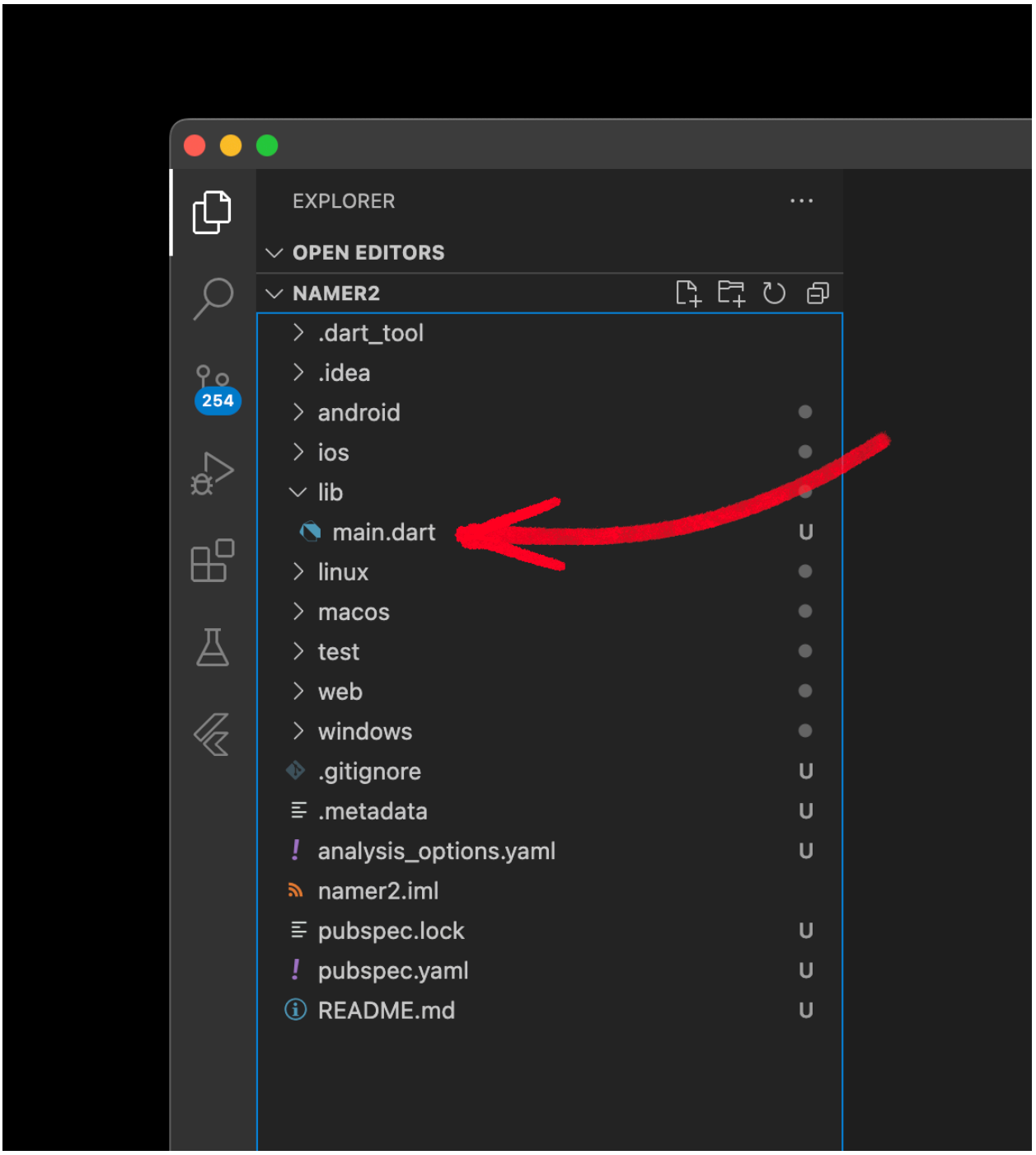
瞭解依附元件

開始探討程式碼前，我們先瞭解為何選擇這些特定套件：

- **built_value**：建立可有效共用記憶體的不可變動物件，對回溯演算法至關重要
- **Riverpod**：提供精細的狀態管理機制，可搭配 `select()` 盡量減少重建次數

- **two_dimensional_scrollables**：處理大型格線時不會影響效能

3. 開啟 `lib/` 目錄中的 `main.dart` 檔案。

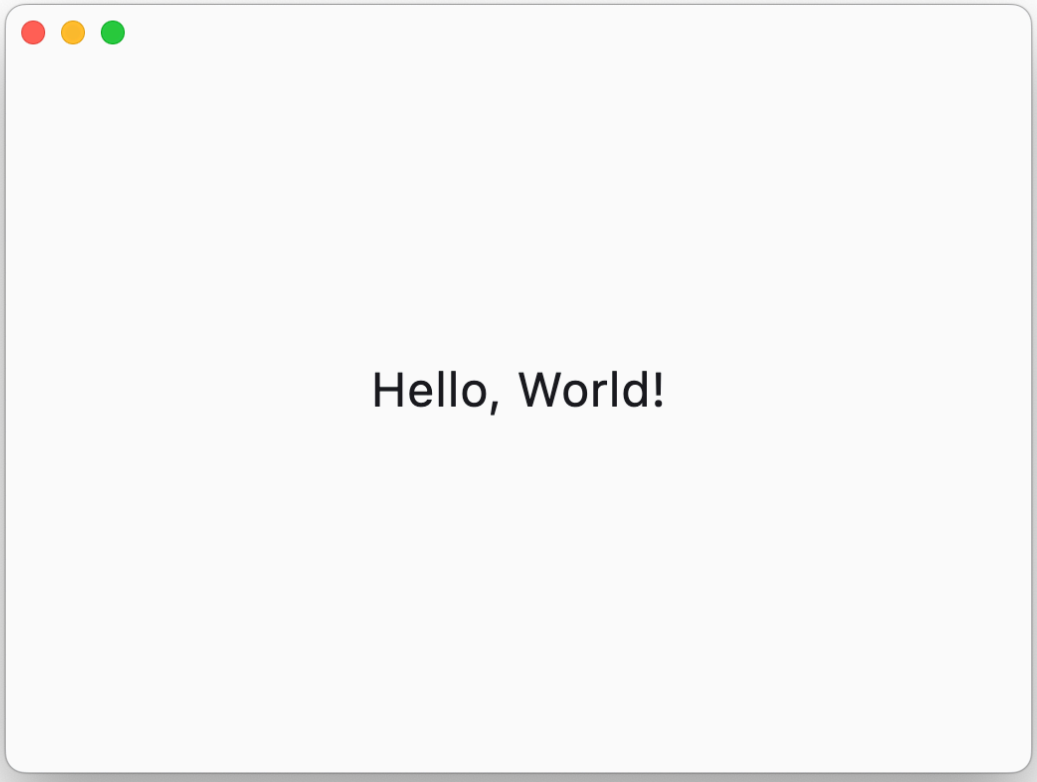


4. 將這個檔案的內容替換成以下內容：

lib/main.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

void main() {
  runApp(
    ProviderScope(
      child: MaterialApp(
        title: 'Crossword Builder',
        debugShowCheckedModeBanner: false,
        theme: ThemeData(
          colorSchemeSeed: Colors.blueGrey,
          brightness: Brightness.light,
        ),
        home: Scaffold(
          body: Center(
            child: Text('Hello, World!', style: TextStyle(fontSize: 24)),
          ),
        ),
      ),
    ),
  );
}
```

5. 執行這段程式碼，確認一切正常。系統應會顯示新視窗，並在各處顯示每個新專案的必要起始片語。其中包含 `ProviderScope`，表示這個應用程式會使用 `riverpod` 進行狀態管理。



檢查點：基本應用程式執行

此時，您應該會看到「Hello, World!」視窗。如果不可以：

- 確認 Flutter 已正確安裝
- 使用 `flutter run` 驗證應用程式是否正常運作
- 確認終端機中沒有編譯錯誤

> **注意**：如要在 macOS 上實現全尺寸內容檢視效果，請修改 `macos/Runner/Base.lproj/MainMenu.xib` XML 檔案，在 `//document/objects/window` XML 節點中加入 `titlebarAppearsTransparent="YES"` 和 `titleVisibility="hidden"` 屬性，並在 `//document/objects/window/windowStyleMask` XML 節點中加入 `fullSizeContentView="YES"` 屬性。如要深入瞭解如何讓 Flutter 應用程式在 macOS 上呈現原生外觀，請參閱 `macos_window_utils` 和 `macos_ui` Flutter 套件。

步驟 3 · 約 5 分鐘

3. 新增字詞

填字遊戲的構成元素

填字遊戲的本質就是字詞清單。這些字詞會排列在格線中，有些是橫向，有些是直向，彼此交錯。解出一個字詞後，就能獲得與該字詞交錯的字詞線索。因此，字詞清單是良好的第一塊基石。

Peter Norvig 的「自然語言語料庫資料」頁面是這些字詞的絕佳來源。SOWPODS 清單是實用的起點，內含 267,750 個字詞。

在這個步驟中，您會下載字詞清單、將其新增為 Flutter 應用程式的資產，並安排 Riverpod 提供者在啟動時將清單載入應用程式。

如要開始，請按照下列步驟操作：

1. 修改專案的 `pubspec.yaml` 檔案，為所選字詞清單新增下列資產宣告。由於其他部分維持不變，因此這個清單只會顯示應用程式設定的 Flutter 節。

pubspec.yaml

```
flutter:
  uses-material-design: true
  assets:                                 # Add this line
    - assets/words.txt                    # And this one.
```

由於您尚未建立這個檔案，編輯器可能會醒目顯示最後一行，並發出警告。

2. 使用瀏覽器和編輯器，在專案頂層建立 `assets` 目錄，並在其中建立 `words.txt` 檔案，其中包含先前連結的其中一個字詞清單。

這個程式碼是根據先前提及的 SOWPODS 清單設計，但應該適用於任何只包含 A-Z 字元的字詞清單。將這個程式碼集擴充為可處理不同字元集，是留給讀者的練習。

載入字詞

如要編寫負責在應用程式啟動時載入字詞清單的程式碼，請按照下列步驟操作：

1. 在 `lib` 目錄中建立 `providers.dart` 檔案。
2. 在檔案中新增下列內容：

lib/providers.dart

```dart
import 'dart:convert';

import 'package:built_collection/built_collection.dart';
import 'package:flutter/services.dart';
import 'package:riverpod/riverpod.dart';
import 'package:riverpod_annotation/riverpod_annotation.dart';

part 'providers.g.dart';

/// A provider for the wordlist to use when generating the crossword.
@riverpod
Future<BuiltSet<String>> wordList(Ref ref) async {
  // This codebase requires that all words consist of lowercase characters
  // in the range 'a'-'z'. Words containing uppercase letters will be
  // lowercased, and words containing runes outside this range will
  // be removed.

  final re = RegExp(r'^[a-z]+$');
  final words = await rootBundle.loadString('assets/words.txt');
  return const LineSplitter()
      .convert(words)
      .toBuiltSet()
      .rebuild(
        (b) => b
          ..map((word) => word.toLowerCase().trim())
          ..where((word) => word.length > 2)
          ..where((word) => re.hasMatch(word)),
      );
}
```

這是程式碼集的第一個 Riverpod 提供者。

> **預期錯誤**：編輯器會以紅色底線標示未定義的檔案 `providers.g.dart`。這是正常現象，這些檔案會在下一個步驟中由 `build_runner` 工具產生。

這項服務的運作方式：

1. 以非同步方式從資產載入字詞清單
2. 篩選字詞，只保留長度超過 2 個字母的 a-z 字元
3. 傳回不可變動的 `BuiltSet`，以利隨機存取

這個專案會為多個依附元件 (包括 Riverpod) 產生程式碼。

3. 如要開始產生程式碼，請執行下列指令：

```
$ dart run build_runner watch -d
[INFO] Generating build script completed, took 174ms
[INFO] Setting up file watchers completed, took 5ms
[INFO] Waiting for all file watchers to be ready completed, took 202ms
[INFO] Reading cached asset graph completed, took 65ms
[INFO] Checking for updates since last build completed, took 680ms
[INFO] Running build completed, took 2.3s
[INFO] Caching finalized dependency graph completed, took 42ms
[INFO] Succeeded after 2.3s with 122 outputs (243 actions)
```

這項程序會在背景繼續執行，並在您變更專案時更新產生的檔案。這個指令在 `providers.g.dart` 中產生程式碼後，編輯器應該會對您新增至 `providers.dart` 的程式碼感到滿意。

在 Riverpod 中，先前定義的 `wordList` 函式等供應商通常會延遲例項化。不過，就這個應用程式而言，您需要積極載入字詞清單。Riverpod 文件建議採用下列方法，處理需要搶先載入的供應商。您現在要實作這項功能。

4. 在 `lib/widgets` 目錄中建立 `crossword_generator_app.dart` 檔案。
5. 在檔案中新增下列內容：

lib/widgets/crossword_generator_app.dart

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../providers.dart';

class CrosswordGeneratorApp extends StatelessWidget {
  const CrosswordGeneratorApp({super.key});

  @override
  Widget build(BuildContext context) {
    return _EagerInitialization(
      child: Scaffold(
        appBar: AppBar(
          titleTextStyle: TextStyle(
            color: Theme.of(context).colorScheme.primary,
            fontSize: 16,
            fontWeight: FontWeight.bold,
          ),
          title: Text('Crossword Generator'),
        ),
        body: SafeArea(
          child: Consumer(
            builder: (context, ref, _) {
              final wordListAsync = ref.watch(wordListProvider);
              return wordListAsync.when(
                data: (wordList) => ListView.builder(
                  itemCount: wordList.length,
                  itemBuilder: (context, index) {
                    return ListTile(title: Text(wordList.elementAt(index)));
                  },
                ),
                error: (error, stackTrace) => Center(child: Text('$error')),
                loading: () => Center(child: CircularProgressIndicator()),
              );
            },
          ),
        ),
      ),
    );
  }
}

class _EagerInitialization extends ConsumerWidget {
  const _EagerInitialization({required this.child});
  final Widget child;

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    ref.watch(wordListProvider);
    return child;
```

```
  }
}
```

這個檔案可從兩個不同方向進行分析。第一個是 `_EagerInitialization` 小工具，其唯一任務是要求先前建立的 `wordList` 提供者載入字詞清單。這個小工具會使用 `ref.watch()` 呼叫監聽提供者，達成這個目標。如要進一步瞭解這項技術，請參閱 Riverpod 說明文件中的「提供者的搶先初始化」一節。

這個檔案中第二個值得注意的重點是 Riverpod 如何處理非同步內容。如您所知，`wordList` 提供者定義為非同步函式，因為從磁碟載入內容的速度較慢。在這個程式碼中，您會收到 `AsyncValue<BuiltSet<String>>`，該型別的 `AsyncValue` 部分是提供者的非同步世界與 Widget `build` 方法的同步世界之間的介面卡。

`AsyncValue` 的 `when` 方法會處理未來值可能處於的三種狀態。Future 可能已順利解決，此時會叫用 `data` 回呼；也可能處於錯誤狀態，此時會叫用 `error` 回呼；最後，也可能仍在載入中。這三個回呼的傳回型別必須相容，因為呼叫的回呼傳回的內容會由 `when` 方法傳回。在本例中，when 方法的結果會顯示為 `Scaffold` 小工具的 `body`。

建立近乎無限的清單應用程式

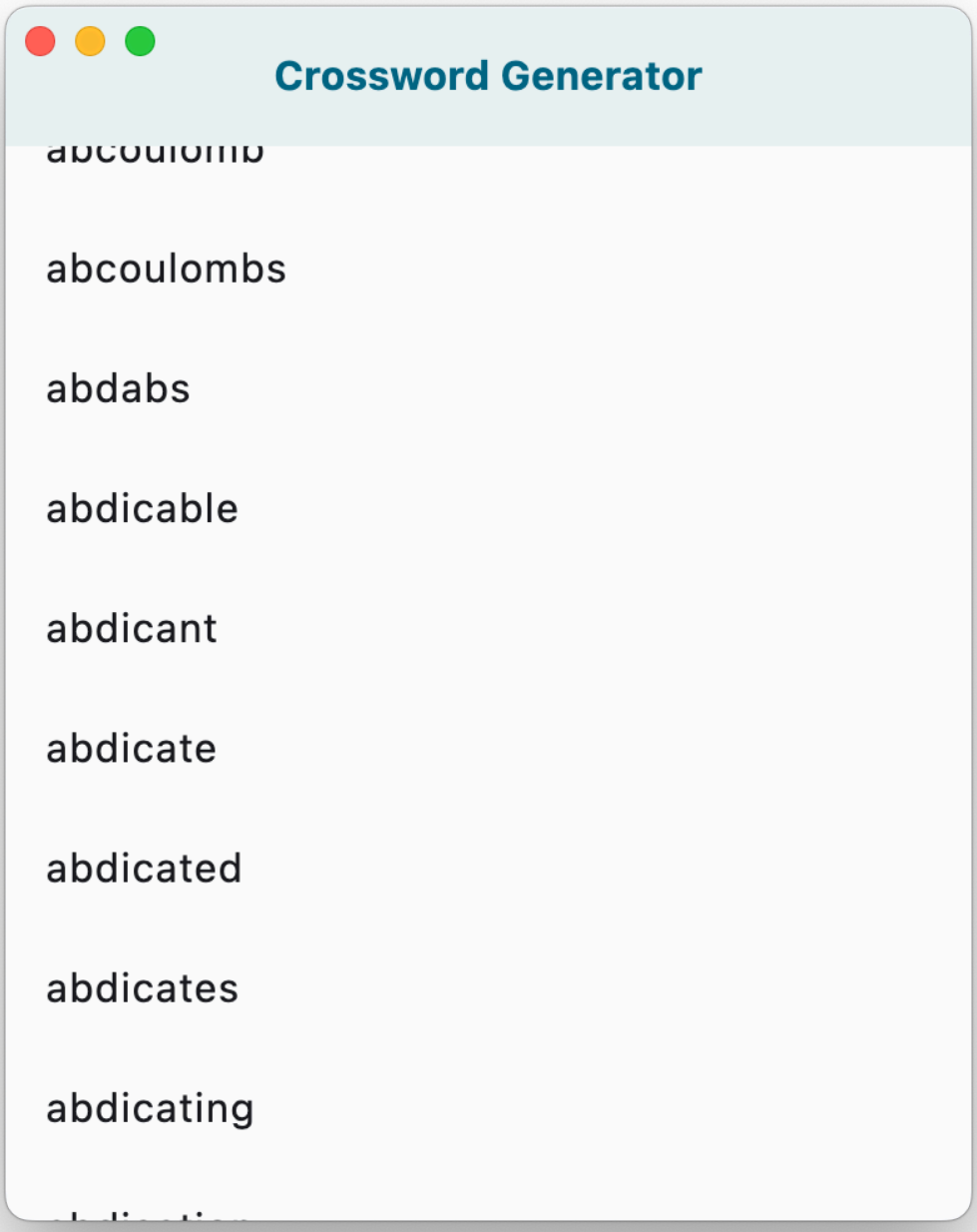
如要在應用程式中整合 `CrosswordGeneratorApp` 小工具，請按照下列步驟操作：

1. 新增下列程式碼，更新 `lib/main.dart` 檔案：

lib/main.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import 'widgets/crossword_generator_app.dart';              // Add this import

void main() {
  runApp(
    ProviderScope(
      child: MaterialApp(
        title: 'Crossword Builder',
        debugShowCheckedModeBanner: false,
        theme: ThemeData(
          colorSchemeSeed: Colors.blueGrey,
          brightness: Brightness.light,
        ),
        home: CrosswordGeneratorApp(),                      // Remove what was here and
replace
      ),
    ),
  );
}
```

2. 重新啟動應用程式。您應該會看到捲動清單，其中包含字典中的所有 267,750 個以上的字詞。



接下來要建構的內容

現在要使用不可變動的物件，為填字遊戲建立核心資料結構。這個基礎可支援有效率的演算法，並順暢更新 UI。

步驟 4 · 約 5 分鐘

4. 以格狀模式顯示字詞

在這個步驟中，您將使用 `built_value` 和 `built_collection` 套件建立資料結構，以便製作填字遊戲。這兩個套件可將資料結構建構為不可變動的值，有助於在 Isolate 之間傳遞資料，並大幅簡化深度優先搜尋和回溯的實作程序。

如要開始，請按照下列步驟操作：

1. 在 `lib` 目錄中建立 `model.dart` 檔案，然後在檔案中加入以下內容：

lib/model.dart

```
import 'package:built_collection/built_collection.dart';
import 'package:built_value/built_value.dart';
import 'package:built_value/serializer.dart';
import 'package:characters/characters.dart';

part 'model.g.dart';

/// A location in a crossword.
abstract class Location implements Built<Location, LocationBuilder> {
  static Serializer<Location> get serializer => _$locationSerializer;

  /// The horizontal part of the location. The location is 0 based.
  int get x;

  /// The vertical part of the location. The location is 0 based.
  int get y;

  /// Returns a new location that is one step to the left of this location.
  Location get left => rebuild((b) => b.x = x - 1);

  /// Returns a new location that is one step to the right of this location.
  Location get right => rebuild((b) => b.x = x + 1);

  /// Returns a new location that is one step up from this location.
  Location get up => rebuild((b) => b.y = y - 1);

  /// Returns a new location that is one step down from this location.
  Location get down => rebuild((b) => b.y = y + 1);

  /// Returns a new location that is [offset] steps to the left of this location.
  Location leftOffset(int offset) => rebuild((b) => b.x = x - offset);

  /// Returns a new location that is [offset] steps to the right of this location.
  Location rightOffset(int offset) => rebuild((b) => b.x = x + offset);

  /// Returns a new location that is [offset] steps up from this location.
  Location upOffset(int offset) => rebuild((b) => b.y = y - offset);

  /// Returns a new location that is [offset] steps down from this location.
  Location downOffset(int offset) => rebuild((b) => b.y = y + offset);

  /// Pretty print a location as a (x,y) coordinate.
  String prettyPrint() => '($x,$y)';

  /// Returns a new location built from [updates]. Both [x] and [y] are
  /// required to be non-null.
  factory Location([void Function(LocationBuilder)? updates]) = _$Location;
  Location._();

  /// Returns a location at the given coordinates.
  factory Location.at(int x, int y) {
    return Location((b) {
```

```
      b
        ..x = x
        ..y = y;
    });
  }
}

/// The direction of a word in a crossword.
enum Direction {
  across,
  down;

  @override
  String toString() => name;
}

/// A word in a crossword. This is a word at a location in a crossword, in either
/// the across or down direction.
abstract class CrosswordWord
    implements Built<CrosswordWord, CrosswordWordBuilder> {
  static Serializer<CrosswordWord> get serializer => _$crosswordWordSerializer;

  /// The word itself.
  String get word;

  /// The location of this word in the crossword.
  Location get location;

  /// The direction of this word in the crossword.
  Direction get direction;

  /// Compare two CrosswordWord by coordinates, x then y.
  static int locationComparator(CrosswordWord a, CrosswordWord b) {
    final compareRows = a.location.y.compareTo(b.location.y);
    final compareColumns = a.location.x.compareTo(b.location.x);
    return switch (compareColumns) {
      0 => compareRows,
      _ => compareColumns,
    };
  }

  /// Constructor for [CrosswordWord].
  factory CrosswordWord.word({
    required String word,
    required Location location,
    required Direction direction,
  }) {
    return CrosswordWord(
      (b) => b
        ..word = word
        ..direction = direction
        ..location.replace(location),
```

```
    );
  }

  /// Constructor for [CrosswordWord].
  /// Use [CrosswordWord.word] instead.
  factory CrosswordWord([void Function(CrosswordWordBuilder)? updates]) =
      _$CrosswordWord;
  CrosswordWord._();
}

/// A character in a crossword. This is a single character at a location in a
/// crossword. It may be part of an across word, a down word, both, but not
/// neither. The neither constraint is enforced elsewhere.
abstract class CrosswordCharacter
    implements Built<CrosswordCharacter, CrosswordCharacterBuilder> {
  static Serializer<CrosswordCharacter> get serializer =>
      _$crosswordCharacterSerializer;

  /// The character at this location.
  String get character;

  /// The across word that this character is a part of.
  CrosswordWord? get acrossWord;

  /// The down word that this character is a part of.
  CrosswordWord? get downWord;

  /// Constructor for [CrosswordCharacter].
  /// [acrossWord] and [downWord] are optional.
  factory CrosswordCharacter.character({
    required String character,
    CrosswordWord? acrossWord,
    CrosswordWord? downWord,
  }) {
    return CrosswordCharacter((b) {
      b.character = character;
      if (acrossWord != null) {
        b.acrossWord.replace(acrossWord);
      }
      if (downWord != null) {
        b.downWord.replace(downWord);
      }
    });
  }

  /// Constructor for [CrosswordCharacter].
  /// Use [CrosswordCharacter.character] instead.
  factory CrosswordCharacter([
    void Function(CrosswordCharacterBuilder)? updates,
  ]) = _$CrosswordCharacter;
  CrosswordCharacter._();
}
```

```
/// A crossword puzzle. This is a grid of characters with words placed in it.
/// The puzzle constraint is in the English crossword puzzle tradition.
abstract class Crossword implements Built<Crossword, CrosswordBuilder> {
  /// Serializes and deserializes the [Crossword] class.
  static Serializer<Crossword> get serializer => _$crosswordSerializer;

  /// Width across the [Crossword] puzzle.
  int get width;

  /// Height down the [Crossword] puzzle.
  int get height;

  /// The words in the crossword.
  BuiltList<CrosswordWord> get words;

  /// The characters by location. Useful for displaying the crossword.
  BuiltMap<Location, CrosswordCharacter> get characters;

  /// Add a word to the crossword at the given location and direction.
  Crossword addWord({
    required Location location,
    required String word,
    required Direction direction,
  }) {
    return rebuild(
      (b) => b
        ..words.add(
          CrosswordWord.word(
            word: word,
            direction: direction,
            location: location,
          ),
        ),
    );
  }

  /// As a finalize step, fill in the characters map.
  @BuiltValueHook(finalizeBuilder: true)
  static void _fillCharacters(CrosswordBuilder b) {
    b.characters.clear();

    for (final word in b.words.build()) {
      for (final (idx, character) in word.word.characters.indexed) {
        switch (word.direction) {
          case Direction.across:
            b.characters.updateValue(
              word.location.rightOffset(idx),
              (b) => b.rebuild((bInner) => bInner.acrossWord.replace(word)),
              ifAbsent: () => CrosswordCharacter.character(
                acrossWord: word,
                character: character,
```

```
            ),
          );
        case Direction.down:
          b.characters.updateValue(
            word.location.downOffset(idx),
            (b) => b.rebuild((bInner) => bInner.downWord.replace(word)),
            ifAbsent: () => CrosswordCharacter.character(
              downWord: word,
              character: character,
            ),
          );
      }
    }
  }
}

/// Pretty print a crossword. Generates the character grid, and lists
/// the down words and across words sorted by location.
String prettyPrintCrossword() {
  final buffer = StringBuffer();
  final grid = List.generate(
    height,
    (_) => List.generate(
      width,
      (_) => '▒', // https://www.compart.com/en/unicode/U+2591
    ),
  );

  for (final MapEntry(key: Location(:x, :y), value: character)
      in characters.entries) {
    grid[y][x] = character.character;
  }

  for (final row in grid) {
    buffer.writeln(row.join());
  }

  buffer.writeln();
  buffer.writeln('Across:');
  for (final word
      in words.where((word) => word.direction == Direction.across).toList()
        ..sort(CrosswordWord.locationComparator)) {
    buffer.writeln('${word.location.prettyPrint()}: ${word.word}');
  }

  buffer.writeln();
  buffer.writeln('Down:');
  for (final word
      in words.where((word) => word.direction == Direction.down).toList()
        ..sort(CrosswordWord.locationComparator)) {
    buffer.writeln('${word.location.prettyPrint()}: ${word.word}');
  }
```

```
    return buffer.toString();
  }

  /// Constructor for [Crossword].
  factory Crossword.crossword({
    required int width,
    required int height,
    Iterable<CrosswordWord>? words,
  }) {
    return Crossword((b) {
      b
        ..width = width
        ..height = height;
      if (words != null) {
        b.words.addAll(words);
      }
    });
  }

  /// Constructor for [Crossword].
  /// Use [Crossword.crossword] instead.
  factory Crossword([void Function(CrosswordBuilder)? updates]) = _$Crossword;
  Crossword._();
}

/// Construct the serialization/deserialization code for the data model.
@SerializersFor([Location, Crossword, CrosswordWord, CrosswordCharacter])
final Serializers serializers = _$serializers;
```

> **注意**：如果您仍有上一步執行的 dart 執行 `build_runner watch -d`，系統會顯示 `model.g.dart` 檔案。如果沒有，現在是重新 `build_runner` 的好時機。

這個檔案說明您將用於建立填字遊戲的資料結構開頭。填字遊戲的本質是水平和垂直字詞的清單，這些字詞會交錯排列在網格中。如要使用這個資料結構，請使用 `Crossword.crossword` 具名建構函式建構適當大小的 `Crossword`，然後使用 `addWord` 方法新增字詞。建構最終值時，`_fillCharacters` 方法會建立 `CrosswordCharacter` 格線。

> **注意**：目前系統不會驗證新增的字詞是否與填字遊戲中已有的字詞相符。我們將在後續步驟修正這個問題。

如要使用這項資料結構，請按照下列步驟操作：

1. 在 `lib` 目錄中建立 `utils` 檔案，然後在檔案中加入以下內容：

lib/utils.dart

```
import 'dart:math';

import 'package:built_collection/built_collection.dart';

/// A [Random] instance for generating random numbers.
final _random = Random();

/// An extension on [BuiltSet] that adds a method to get a random element.
extension RandomElements<E> on BuiltSet<E> {
  E randomElement() {
    return elementAt(_random.nextInt(length));
  }
}
```

這是 `BuiltSet` 的擴充功能，可輕鬆擷取集合的隨機元素。擴充方法是擴充類別並新增功能的絕佳方式。如要在 `utils.dart` 檔案以外使用擴充功能，必須為擴充功能命名。

2. 在 `lib/providers.dart` 檔案中新增下列匯入項目：

lib/providers.dart

```
import 'dart:convert';
import 'dart:math';                                     // Add this import

import 'package:built_collection/built_collection.dart';
import 'package:flutter/foundation.dart';               // Add this import
import 'package:flutter/services.dart';
import 'package:riverpod_annotation/riverpod_annotation.dart';

import 'model.dart' as model;                           // And this import
import 'utils.dart';                                    // And this one

part 'providers.g.dart';

/// A provider for the wordlist to use when generating the crossword.
@riverpod
Future<BuiltSet<String>> wordList(WordListRef ref) async {
```

這些匯入作業會將先前定義的模型公開給您即將建立的供應器。`dart:math` 匯入內容適用於 `Random`，`flutter/foundation.dart` 匯入內容適用於 `debugPrint`，`model.dart` 適用於模型，而 `utils.dart` 適用於 `BuiltSet` 擴充功能。

3. 在同一個檔案的結尾處，新增下列供應器：

lib/providers.dart

```
/// An enumeration for different sizes of [model.Crossword]s.
enum CrosswordSize {
  small(width: 20, height: 11),
  medium(width: 40, height: 22),
  large(width: 80, height: 44),
  xlarge(width: 160, height: 88),
  xxlarge(width: 500, height: 500);

  const CrosswordSize({required this.width, required this.height});

  final int width;
  final int height;
  String get label => '$width x $height';
}

/// A provider that holds the current size of the crossword to generate.
@Riverpod(keepAlive: true)
class Size extends _$Size {
  var _size = CrosswordSize.medium;

  @override
  CrosswordSize build() => _size;

  void setSize(CrosswordSize size) {
    _size = size;
    ref.invalidateSelf();
  }
}

final _random = Random();

@riverpod
Stream<model.Crossword> crossword(Ref ref) async* {
  final size = ref.watch(sizeProvider);
  final wordListAsync = ref.watch(wordListProvider);

  var crossword = model.Crossword.crossword(
    width: size.width,
    height: size.height,
  );

  yield* wordListAsync.when(
    data: (wordList) async* {
      while (crossword.characters.length < size.width * size.height * 0.8) {
        final word = wordList.randomElement();
        final direction = _random.nextBool()
            ? model.Direction.across
            : model.Direction.down;
        final location = model.Location.at(
          _random.nextInt(size.width),
          _random.nextInt(size.height),
        );
```

```
        crossword = crossword.addWord(
          word: word,
          direction: direction,
          location: location,
        );
        yield crossword;
        await Future.delayed(Duration(milliseconds: 100));
      }

      yield crossword;
    },
    error: (error, stackTrace) async* {
      debugPrint('Error loading word list: $error');
      yield crossword;
    },
    loading: () async* {
      yield crossword;
    },
  );
}
```

這些變更會在應用程式中新增兩個供應商。第一個是 `Size`，這實際上是包含 `CrosswordSize` 列舉所選值的全域變數。這樣一來，使用者介面就能顯示及設定建構中的填字遊戲大小。第二個供應商 `crossword` 則更具創意。這項函式會傳回一系列 `Crossword`。這項功能是使用 Dart 對產生器的支援功能建構而成，函式上會標示 `async*`。也就是說，這項函式不會在傳回值時結束，而是會產生一系列 `Crossword`，讓您更輕鬆地編寫可傳回中繼結果的運算。

由於 `crossword` 提供者函式開頭有一對 `ref.watch` 呼叫，因此每當填字遊戲所選大小變更，以及字詞清單載入完成時，Riverpod 系統都會重新啟動 `Crossword` 串流。

現在您已擁有產生填字遊戲的程式碼 (雖然都是隨機字詞)，但最好還是向工具使用者顯示這些字詞。

4. 在 `lib/widgets` 目錄中建立 `crossword_widget.dart` 檔案，並加入以下內容：

lib/widgets/crossword_widget.dart

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:two_dimensional_scrollables/two_dimensional_scrollables.dart';

import '../model.dart';
import '../providers.dart';

class CrosswordWidget extends ConsumerWidget {
  const CrosswordWidget({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final size = ref.watch(sizeProvider);
    return TableView.builder(
      diagonalDragBehavior: DiagonalDragBehavior.free,
      cellBuilder: _buildCell,
      columnCount: size.width,
      columnBuilder: (index) => _buildSpan(context, index),
      rowCount: size.height,
      rowBuilder: (index) => _buildSpan(context, index),
    );
  }

  TableViewCell _buildCell(BuildContext context, TableVicinity vicinity) {
    final location = Location.at(vicinity.column, vicinity.row);

    return TableViewCell(
      child: Consumer(
        builder: (context, ref, _) {
          final character = ref.watch(
            crosswordProvider.select(
              (crosswordAsync) => crosswordAsync.when(
                data: (crossword) => crossword.characters[location],
                error: (error, stackTrace) => null,
                loading: () => null,
              ),
            ),
          );

          if (character != null) {
            return Container(
              color: Theme.of(context).colorScheme.onPrimary,
              child: Center(
                child: Text(
                  character.character,
                  style: TextStyle(
                    fontSize: 24,
                    color: Theme.of(context).colorScheme.primary,
                  ),
                ),
              ),
            );
```

```
          }

          return ColoredBox(
            color: Theme.of(context).colorScheme.primaryContainer,
          );
        },
      ),
    );
  }

  TableSpan _buildSpan(BuildContext context, int index) {
    return TableSpan(
      extent: FixedTableSpanExtent(32),
      foregroundDecoration: TableSpanDecoration(
        border: TableSpanBorder(
          leading: BorderSide(
            color: Theme.of(context).colorScheme.onPrimaryContainer,
          ),
          trailing: BorderSide(
            color: Theme.of(context).colorScheme.onPrimaryContainer,
          ),
        ),
      ),
    );
  }
}
```

這個小工具是 `ConsumerWidget`，可以直接依賴 `Size` 提供者判斷格線大小，以顯示 `Crossword` 的字元。這個格線的顯示方式是透過 `two_dimensional_scrollables` 套件中的 `TableView` 小工具達成。

值得注意的是，`_buildCell` 輔助函式轉譯的每個儲存格，都會在傳回的 `Widget` 樹狀結構中包含 `Consumer` 小工具。這會做為重新整理的界線。當 `ref.watch` 的傳回值變更時，`Consumer` 小工具內的所有項目都會重新建立。每當 `Crossword` 變更時，您可能會想重新建立整個樹狀結構，但這樣會導致大量運算，而使用這項設定即可略過這些運算。

如果您查看 `ref.watch` 的參數，會發現系統使用 `crosswordProvider.select`，避免重新計算版面配置，因此有另一層的迴避措施。也就是說，只有在儲存格負責算繪的字元變更時，`ref.watch` 才會觸發 `TableViewCell` 內容的重建作業。減少重新算繪次數是確保 UI 回應速度的必要做法。

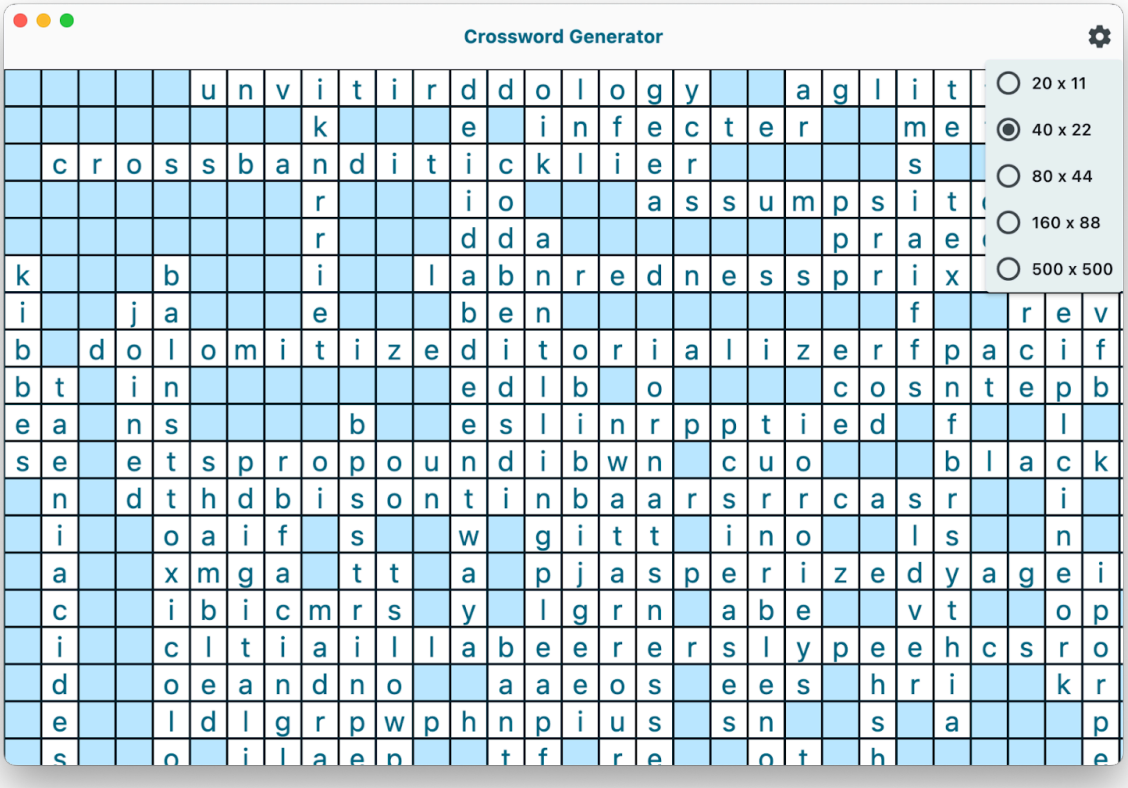
如要向使用者公開 `CrosswordWidget` 和 `Size` 提供者，請按照下列方式變更 `crossword_generator_app.dart` 檔案：

lib/widgets/crossword_generator_app.dart

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../providers.dart';
import 'crossword_widget.dart';                                  // Add this import

class CrosswordGeneratorApp extends StatelessWidget {
  const CrosswordGeneratorApp({super.key});

  @override
  Widget build(BuildContext context) {
    return _EagerInitialization(
      child: Scaffold(
        appBar: AppBar(
          actions: [_CrosswordGeneratorMenu()],                  // Add this line
          titleTextStyle: TextStyle(
            color: Theme.of(context).colorScheme.primary,
            fontSize: 16,
            fontWeight: FontWeight.bold,
          ),
          title: Text('Crossword Generator'),
        ),
        body: SafeArea(child: CrosswordWidget()),                // Replace what was here before
      ),
    );
  }
}

class _EagerInitialization extends ConsumerWidget {
  const _EagerInitialization({required this.child});
  final Widget child;

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    ref.watch(wordListProvider);
    return child;
  }
}

class _CrosswordGeneratorMenu extends ConsumerWidget {          // Add from here
  @override
  Widget build(BuildContext context, WidgetRef ref) => MenuAnchor(
    menuChildren: [
      for (final entry in CrosswordSize.values)
        MenuItemButton(
          onPressed: () => ref.read(sizeProvider.notifier).setSize(entry),
          leadingIcon: entry == ref.watch(sizeProvider)
              ? Icon(Icons.radio_button_checked_outlined)
              : Icon(Icons.radio_button_unchecked_outlined),
          child: Text(entry.label),
        ),
    ],
```

```
    builder: (context, controller, child) => IconButton(
      onPressed: () => controller.open(),
      icon: Icon(Icons.settings),
    ),
  );                                                    // To here.
}
```

這裡有幾項變更。首先，負責將 `wordList` 算繪為 `ListView` 的程式碼已替換為對 `lib/widgets/crossword_widget.dart` 檔案中定義的 `CrosswordWidget` 的呼叫。另一項重大變更，是開始提供可變更應用程式行為的選單，首先是變更填字遊戲的大小。後續步驟會新增更多 `MenuItemButton`。執行應用程式，畫面應如下所示：



格線中會顯示字元，使用者可以透過選單變更格線大小。但這些字詞並非以填字遊戲的形式排列。這是因為系統未對字詞加入填字遊戲的方式強制執行任何限制。簡單來說，就是一團亂。您將在下一個步驟中開始控管這些項目！

步驟 5 · 約 5 分鐘

5. 強制執行限制

異動內容和原因

目前你的填字遊戲允許重疊的字詞，且沒有驗證機制。您將新增限制檢查，確保字詞能像真正的填字遊戲一樣正確交錯。

本步驟的目標是在模型中加入程式碼，強制執行填字遊戲限制。填字遊戲的類型有很多種，本程式碼研究室將遵循傳統的英文填字遊戲風格。如要修改這段程式碼來生成其他樣式的填字遊戲，請讀者自行練習。

如要開始，請按照下列步驟操作：

1. 開啟 `model.dart` 檔案，並將 `Crossword` 模型替換為下列模型：

lib/model.dart

```
/// A crossword puzzle. This is a grid of characters with words placed in it.
/// The puzzle constraint is in the English crossword puzzle tradition.
abstract class Crossword implements Built<Crossword, CrosswordBuilder> {
  /// Serializes and deserializes the [Crossword] class.
  static Serializer<Crossword> get serializer => _$crosswordSerializer;

  /// Width across the [Crossword] puzzle.
  int get width;

  /// Height down the [Crossword] puzzle.
  int get height;

  /// The words in the crossword.
  BuiltList<CrosswordWord> get words;

  /// The characters by location. Useful for displaying the crossword,
  /// or checking the proposed solution.
  BuiltMap<Location, CrosswordCharacter> get characters;

  /// Checks if this crossword is valid.
  bool get valid {
    // Check that there are no duplicate words.
    final wordSet = words.map((word) => word.word).toBuiltSet();
    if (wordSet.length != words.length) {
      return false;
    }

    for (final MapEntry(key: location, value: character)
        in characters.entries) {
      // All characters must be a part of an across or down word.
      if (character.acrossWord == null && character.downWord == null) {
        return false;
      }

      // All characters must be within the crossword puzzle.
      // No drawing outside the lines.
      if (location.x < 0 ||
          location.y < 0 ||
          location.x >= width ||
          location.y >= height) {
        return false;
      }

      // Characters above and below this character must be related
      // by a vertical word
      if (characters[location.up] case final up?) {
        if (character.downWord == null) {
          return false;
        }
        if (up.downWord != character.downWord) {
          return false;
        }
```

```
    }

    if (characters[location.down] case final down?) {
      if (character.downWord == null) {
        return false;
      }
      if (down.downWord != character.downWord) {
        return false;
      }
    }

    // Characters to the left and right of this character must be
    // related by a horizontal word
    final left = characters[location.left];
    if (left != null) {
      if (character.acrossWord == null) {
        return false;
      }
      if (left.acrossWord != character.acrossWord) {
        return false;
      }
    }

    final right = characters[location.right];
    if (right != null) {
      if (character.acrossWord == null) {
        return false;
      }
      if (right.acrossWord != character.acrossWord) {
        return false;
      }
    }
  }

  return true;
}

/// Add a word to the crossword at the given location and direction.
Crossword? addWord({
  required Location location,
  required String word,
  required Direction direction,
}) {
  // Require that the word is not already in the crossword.
  if (words.map((crosswordWord) => crosswordWord.word).contains(word)) {
    return null;
  }

  final wordCharacters = word.characters;
  bool overlap = false;

  // Check that the word fits in the crossword.
```

```
  for (final (index, character) in wordCharacters.indexed) {
    final characterLocation = switch (direction) {
      Direction.across => location.rightOffset(index),
      Direction.down => location.downOffset(index),
    };

    final target = characters[characterLocation];
    if (target != null) {
      overlap = true;
      if (target.character != character) {
        return null;
      }
      if (direction == Direction.across && target.acrossWord != null ||
          direction == Direction.down && target.downWord != null) {
        return null;
      }
    }
  }
  if (words.isNotEmpty && !overlap) {
    return null;
  }

  final candidate = rebuild(
    (b) => b
      ..words.add(
        CrosswordWord.word(
          word: word,
          direction: direction,
          location: location,
        ),
      ),
  );

  if (candidate.valid) {
    return candidate;
  } else {
    return null;
  }
}

/// As a finalize step, fill in the characters map.
@BuiltValueHook(finalizeBuilder: true)
static void _fillCharacters(CrosswordBuilder b) {
  b.characters.clear();

  for (final word in b.words.build()) {
    for (final (idx, character) in word.word.characters.indexed) {
      switch (word.direction) {
        case Direction.across:
          b.characters.updateValue(
            word.location.rightOffset(idx),
            (b) => b.rebuild((bInner) => bInner.acrossWord.replace(word)),
```

```
            ifAbsent: () => CrosswordCharacter.character(
              acrossWord: word,
              character: character,
            ),
          );
        case Direction.down:
          b.characters.updateValue(
            word.location.downOffset(idx),
            (b) => b.rebuild((bInner) => bInner.downWord.replace(word)),
            ifAbsent: () => CrosswordCharacter.character(
              downWord: word,
              character: character,
            ),
          );
      }
    }
  }
}

/// Pretty print a crossword. Generates the character grid, and lists
/// the down words and across words sorted by location.
String prettyPrintCrossword() {
  final buffer = StringBuffer();
  final grid = List.generate(
    height,
    (_) => List.generate(
      width,
      (_) => '▒', // https://www.compart.com/en/unicode/U+2591
    ),
  );

  for (final MapEntry(key: Location(:x, :y), value: character)
      in characters.entries) {
    grid[y][x] = character.character;
  }

  for (final row in grid) {
    buffer.writeln(row.join());
  }

  buffer.writeln();
  buffer.writeln('Across:');
  for (final word
      in words.where((word) => word.direction == Direction.across).toList()
        ..sort(CrosswordWord.locationComparator)) {
    buffer.writeln('${word.location.prettyPrint()}: ${word.word}');
  }

  buffer.writeln();
  buffer.writeln('Down:');
  for (final word
      in words.where((word) => word.direction == Direction.down).toList()
```

```
          ..sort(CrosswordWord.locationComparator)) {
      buffer.writeln('${word.location.prettyPrint()}: ${word.word}');
    }

    return buffer.toString();
  }

  /// Constructor for [Crossword].
  factory Crossword.crossword({
    required int width,
    required int height,
    Iterable<CrosswordWord>? words,
  }) {
    return Crossword((b) {
      b
        ..width = width
        ..height = height;
      if (words != null) {
        b.words.addAll(words);
      }
    });
  }

  /// Constructor for [Crossword].
  /// Use [Crossword.crossword] instead.
  factory Crossword([void Function(CrosswordBuilder)? updates]) = _$Crossword;
  Crossword._();
}
```

提醒您，對 `model.dart` 和 `providers.dart` 檔案所做的變更，需要執行 `build_runner` 才能更新對應的 `model.g.dart` 和 `providers.g.dart` 檔案。如果這些檔案尚未自動更新，現在是使用 `dart run build_runner watch -d` 重新啟動 `build_runner` 的好時機。

如要在模型層使用這項新功能，請更新相應的供應商層。

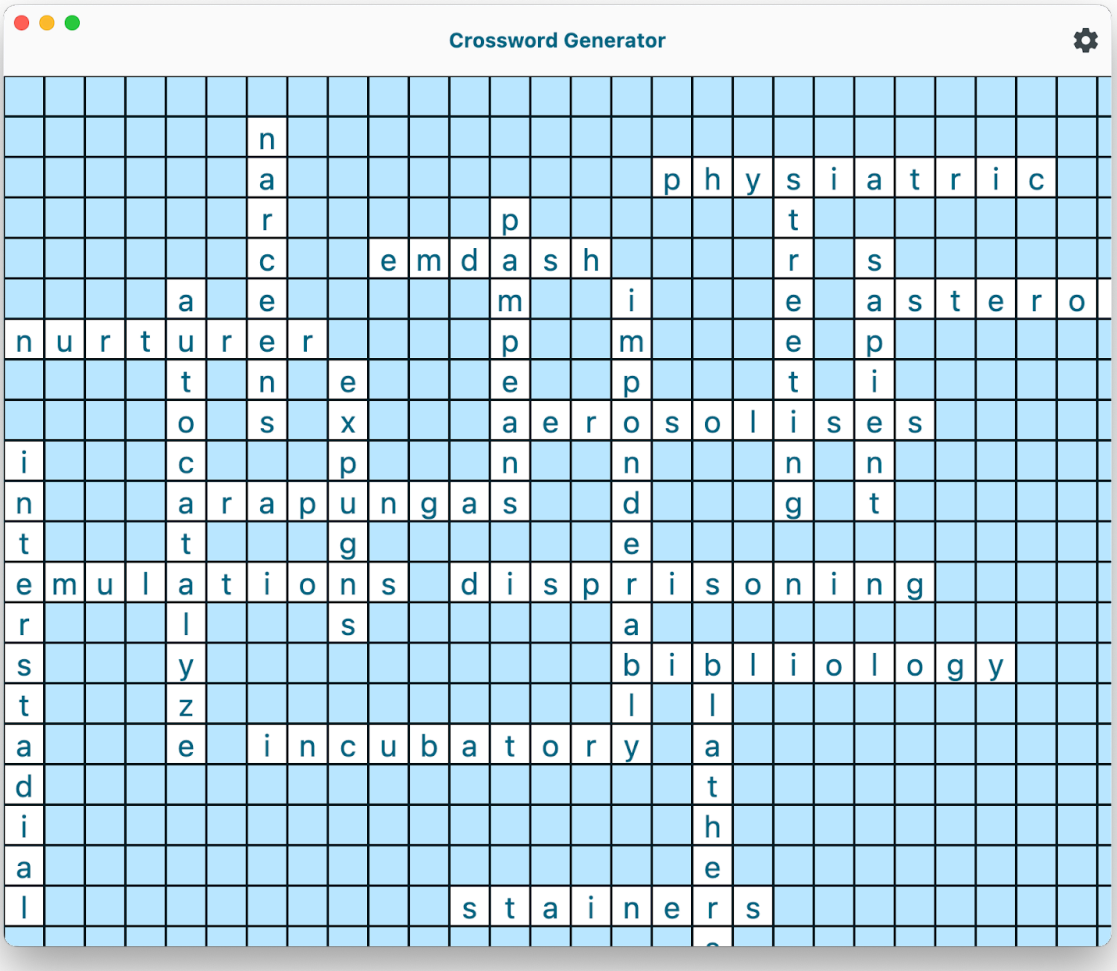
2. 按照下列方式編輯 `providers.dart` 檔案：

lib/providers.dart

```dart
import 'dart:convert';
import 'dart:math';

import 'package:built_collection/built_collection.dart';
import 'package:flutter/foundation.dart';
import 'package:flutter/services.dart';
import 'package:riverpod_annotation/riverpod_annotation.dart';

import 'model.dart' as model;
import 'utils.dart';

part 'providers.g.dart';

/// A provider for the wordlist to use when generating the crossword.
@riverpod
Future<BuiltSet<String>> wordList(WordListRef ref) async {
  // This codebase requires that all words consist of lowercase characters
  // in the range 'a'-'z'. Words containing uppercase letters will be
  // lowercased, and words containing runes outside this range will
  // be removed.

  final re = RegExp(r'^[a-z]+$');
  final words = await rootBundle.loadString('assets/words.txt');
  return const LineSplitter().convert(words).toBuiltSet().rebuild((b) => b
    ..map((word) => word.toLowerCase().trim())
    ..where((word) => word.length > 2)
    ..where((word) => re.hasMatch(word)));
}

/// An enumeration for different sizes of [model.Crossword]s.
enum CrosswordSize {
  small(width: 20, height: 11),
  medium(width: 40, height: 22),
  large(width: 80, height: 44),
  xlarge(width: 160, height: 88),
  xxlarge(width: 500, height: 500);

  const CrosswordSize({
    required this.width,
    required this.height,
  });

  final int width;
  final int height;
  String get label => '$width x $height';
}

/// A provider that holds the current size of the crossword to generate.
@Riverpod(keepAlive: true)
class Size extends _$Size {
  var _size = CrosswordSize.medium;
```

```
  @override
  CrosswordSize build() => _size;

  void setSize(CrosswordSize size) {
    _size = size;
    ref.invalidateSelf();
  }
}

final _random = Random();

@riverpod
Stream<model.Crossword> crossword(CrosswordRef ref) async* {
  final size = ref.watch(sizeProvider);
  final wordListAsync = ref.watch(wordListProvider);

  var crossword =
      model.Crossword.crossword(width: size.width, height: size.height);

  yield* wordListAsync.when(
    data: (wordList) async* {
      while (crossword.characters.length < size.width * size.height * 0.8) {
        final word = wordList.randomElement();
        final direction =
            _random.nextBool() ? model.Direction.across : model.Direction.down;
        final location = model.Location.at(
            _random.nextInt(size.width), _random.nextInt(size.height));

        var candidate = crossword.addWord(                 // Edit from here
            word: word, direction: direction, location: location);
        await Future.delayed(Duration(milliseconds: 10));
        if (candidate != null) {
          debugPrint('Added word: $word');
          crossword = candidate;
          yield crossword;
        } else {
          debugPrint('Failed to add word: $word');
        }                                                  // To here.
      }

      yield crossword;
    },
    error: (error, stackTrace) async* {
      debugPrint('Error loading word list: $error');
      yield crossword;
    },
    loading: () async* {
      yield crossword;
    },
  );
}
```

3. 執行應用程式。UI 中不會發生太多事，但查看記錄時，您會發現許多事件。



如果思考一下這裡發生的情況，我們會發現填字遊戲是隨機出現。`Crossword` 模型中的 `addWord` 方法會拒絕任何不適合目前填字遊戲的建議字詞，因此我們能看到任何字詞出現，都令人驚豔。

為何要改用背景處理？

生成填字遊戲時，您可能會發現 UI 沒有回應。這是因為生成填字遊戲需要進行數千次驗證檢查。這些計算會封鎖 Flutter 的 60 fps 算繪迴圈，因此您會將大量運算作業移至背景隔離區。這樣的好處是，謎題在背景生成時，使用者介面仍能保持流暢

為了更系統化地選擇要嘗試的字詞，將這項運算作業從 UI 執行緒移至背景隔離區會很有幫助。Flutter 提供非常實用的包裝函式，可擷取一連串作業，並在背景隔離區中執行，也就是 `compute` 函式。

4. 在 `providers.dart` 檔案中，按照下列方式修改填字遊戲供應商：

lib/providers.dart

```
@riverpod
Stream<model.Crossword> crossword(Ref ref) async* {
  final size = ref.watch(sizeProvider);
  final wordListAsync = ref.watch(wordListProvider);

  var crossword = model.Crossword.crossword(
    width: size.width,
    height: size.height,
  );

  yield* wordListAsync.when(
    data: (wordList) async* {
      while (crossword.characters.length < size.width * size.height * 0.8) {
        final word = wordList.randomElement();
        final direction = _random.nextBool()
            ? model.Direction.across
            : model.Direction.down;
        final location = model.Location.at(
          _random.nextInt(size.width),
          _random.nextInt(size.height),
        );
        try {                                                  // Edit from here
          var candidate = await compute((
            (String, model.Direction, model.Location) wordToAdd,
          ) {
            final (word, direction, location) = wordToAdd;
            return crossword.addWord(
              word: word,
              direction: direction,
              location: location,
            );
          }, (word, direction, location));

          if (candidate != null) {
            crossword = candidate;
            yield crossword;
          }
        } catch (e) {
          debugPrint('Error running isolate: $e');
        }                                                      // To here.
      }

      yield crossword;
    },
    error: (error, stackTrace) async* {
      debugPrint('Error loading word list: $error');
      yield crossword;
    },
    loading: () async* {
      yield crossword;
    },
```

```
  );
}
```

瞭解隔離限制

這段程式碼可以運作，但隱藏著問題。隔離區有嚴格的規則，規定哪些資料可以在隔離區之間傳遞，問題在於閉包「擷取」了提供者參照，而該參照無法序列化並傳送至其他隔離區。

系統嘗試傳送無法序列化的資料時，會顯示以下訊息：

```
flutter: Error running isolate: Invalid argument(s): Illegal argument in isolate
message: object is unsendable - Library:'dart:async' Class: _Future@4048458 (see
restrictions listed at `SendPort.send()` documentation for more information)
flutter:  <- Instance of 'AutoDisposeStreamProviderElement<Crossword>' (from
package:riverpod/src/stream_provider.dart)
flutter:  <- Context num_variables: 2 <- Context num_variables: 1 parent:{ Context
num_variables: 2 }
flutter:  <- Context num_variables: 1 parent:{ Context num_variables: 1 parent:{
Context num_variables: 2 } }
flutter:  <- Closure: () => Crossword? (from
package:generate_crossword/providers.dart)
```

這是 `compute` 交給背景獨立項目關閉提供者的結果，無法透過 `SendPort.send()` 傳送。如要修正這個問題，請確保關閉的項目沒有任何無法傳送的內容。

第一步是將供應商與 Isolate 程式碼分開。

5. 在 `lib` 目錄中建立 `isolates.dart` 檔案，然後加入以下內容：

lib/isolates.dart

```
import 'dart:math';

import 'package:built_collection/built_collection.dart';
import 'package:flutter/foundation.dart';

import 'model.dart';
import 'utils.dart';

final _random = Random();

Stream<Crossword> exploreCrosswordSolutions({
  required Crossword crossword,
  required BuiltSet<String> wordList,
}) async* {
  while (crossword.characters.length <
      crossword.width * crossword.height * 0.8) {
    final word = wordList.randomElement();
    final direction = _random.nextBool() ? Direction.across : Direction.down;
    final location = Location.at(
      _random.nextInt(crossword.width),
      _random.nextInt(crossword.height),
    );
    try {
      var candidate = await compute(((String, Direction, Location) wordToAdd) {
        final (word, direction, location) = wordToAdd;
        return crossword.addWord(
          word: word,
          direction: direction,
          location: location,
        );
      }, (word, direction, location));

      if (candidate != null) {
        crossword = candidate;
        yield crossword;
      }
    } catch (e) {
      debugPrint('Error running isolate: $e');
    }
  }
}
```

這個程式碼應該相當眼熟。這是 `crossword` 提供者的核心，但現在是獨立的產生器函式。現在您可以更新 `providers.dart` 檔案，使用這個新函式例項化背景隔離區。

lib/providers.dart

```
// Drop the dart:math import, the _random instance moved to isolates.dart
import 'dart:convert';

import 'package:built_collection/built_collection.dart';
import 'package:flutter/foundation.dart';
import 'package:flutter/services.dart';
import 'package:riverpod/riverpod.dart';
import 'package:riverpod_annotation/riverpod_annotation.dart';

import 'isolates.dart';                                    // Add this import
import 'model.dart' as model;
                                                           // Drop the utils.dart import

part 'providers.g.dart';

/// A provider for the wordlist to use when generating the crossword.
@riverpod
Future<BuiltSet<String>> wordList(Ref ref) async {
  // This codebase requires that all words consist of lowercase characters
  // in the range 'a'-'z'. Words containing uppercase letters will be
  // lowercased, and words containing runes outside this range will
  // be removed.

  final re = RegExp(r'^[a-z]+$');
  final words = await rootBundle.loadString('assets/words.txt');
  return const LineSplitter()
      .convert(words)
      .toBuiltSet()
      .rebuild(
        (b) => b
          ..map((word) => word.toLowerCase().trim())
          ..where((word) => word.length > 2)
          ..where((word) => re.hasMatch(word)),
      );
}

/// An enumeration for different sizes of [model.Crossword]s.
enum CrosswordSize {
  small(width: 20, height: 11),
  medium(width: 40, height: 22),
  large(width: 80, height: 44),
  xlarge(width: 160, height: 88),
  xxlarge(width: 500, height: 500);

  const CrosswordSize({required this.width, required this.height});

  final int width;
  final int height;
  String get label => '$width x $height';
}

/// A provider that holds the current size of the crossword to generate.
```

```
@Riverpod(keepAlive: true)
class Size extends _$Size {
  var _size = CrosswordSize.medium;

  @override
  CrosswordSize build() => _size;

  void setSize(CrosswordSize size) {
    _size = size;
    ref.invalidateSelf();
  }
}
                                                        // Drop the _random instance
@riverpod
Stream<model.Crossword> crossword(Ref ref) async* {
  final size = ref.watch(sizeProvider);
  final wordListAsync = ref.watch(wordListProvider);

  final emptyCrossword = model.Crossword.crossword(        // Edit from here
    width: size.width,
    height: size.height,
  );

  yield* wordListAsync.when(
    data: (wordList) => exploreCrosswordSolutions(
      crossword: emptyCrossword,
      wordList: wordList,
    ),
    error: (error, stackTrace) async* {
      debugPrint('Error loading word list: $error');
      yield emptyCrossword;
    },
    loading: () async* {
      yield emptyCrossword;                              // To here.
    },
  );
}
```

現在您可以使用這項工具建立不同大小的填字遊戲，並在背景隔離區中`compute`解謎。現在，如果程式碼在決定要嘗試將哪些字詞加入填字遊戲時，能更有效率就好了。

步驟 6 · 約 5 分鐘

6. 管理工作佇列

瞭解搜尋策略

生成字謎時，系統會使用**回溯**，也就是有系統地反覆試驗。應用程式會先嘗試在某個位置放置單字，然後檢查是否與現有單字相符。如果可以，請保留並嘗試下一個字。如果沒有，請拔除安全金鑰，然後嘗試其他位置。

填字遊戲適用於回溯，因為每個字的位置都會為後續的字建立限制，系統會快速偵測並捨棄無效的位置。不可變更的資料結構可有效「復原」變更。

目前程式碼的問題在於，要解決的問題實際上是搜尋問題，但目前的解決方案是盲目搜尋。如果程式碼專注於尋找可附加至目前字詞的字詞，而不是隨機嘗試將字詞放在格線上的任何位置，系統就能更快找到解決方案。其中一種做法是導入地點的工作佇列，嘗試尋找字詞。

程式碼會建構候選解決方案、檢查候選解決方案是否有效，並視有效性納入候選解決方案或捨棄。這是回溯演算法系列的實作範例。`built_value` 和 `built_collection` 可建立衍生自不可變動值的新不可變動值，因此與衍生來源共用常見狀態，大幅簡化了這項實作程序。這樣一來，就能以低廉的成本開發潛在候選項目，不必負擔深層複製所需的記憶體成本。

> **注意**：什麼是回溯？根據維基百科，回溯演算法是一類演算法，用於尋找某些運算問題的解決方案，特別是限制滿足問題，會逐步建構解決方案的候選項目，並在判斷候選項目不可能完成為有效解決方案時，放棄候選項目或回溯。詳情請參閱「回溯」。

如要開始，請按照下列步驟操作：

1. 開啟 `model.dart` 檔案，並在其中新增下列 `WorkQueue` 定義：

lib/model.dart

```
  /// Constructor for [Crossword].
  /// Use [Crossword.crossword] instead.
  factory Crossword([void Function(CrosswordBuilder)? updates]) = _$Crossword;
  Crossword._();
}
                                                          // Add from here
/// A work queue for a worker to process. The work queue contains a crossword
/// and a list of locations to try, along with candidate words to add to the
/// crossword.
abstract class WorkQueue implements Built<WorkQueue, WorkQueueBuilder> {
  static Serializer<WorkQueue> get serializer => _$workQueueSerializer;

  /// The crossword the worker is working on.
  Crossword get crossword;

  /// The outstanding queue of locations to try.
  BuiltMap<Location, Direction> get locationsToTry;

  /// Known bad locations.
  BuiltSet<Location> get badLocations;

  /// The list of unused candidate words that can be added to this crossword.
  BuiltSet<String> get candidateWords;

  /// Returns true if the work queue is complete.
  bool get isCompleted => locationsToTry.isEmpty || candidateWords.isEmpty;

  /// Create a work queue from a crossword.
  static WorkQueue from({
    required Crossword crossword,
    required Iterable<String> candidateWords,
    required Location startLocation,
  }) => WorkQueue((b) {
    if (crossword.words.isEmpty) {
      // Strip candidate words too long to fit in the crossword
      b.candidateWords.addAll(
        candidateWords.where(
          (word) => word.characters.length <= crossword.width,
        ),
      );

      b.crossword.replace(crossword);

      b.locationsToTry.addAll({startLocation: Direction.across});
    } else {
      // Assuming words have already been stripped to length
      b.candidateWords.addAll(
        candidateWords.toBuiltSet().rebuild(
          (b) => b.removeAll(crossword.words.map((word) => word.word)),
        ),
      );
      b.crossword.replace(crossword);
```

```
    crossword.characters
        .rebuild(
          (b) => b.removeWhere((location, character) {
            if (character.acrossWord != null && character.downWord != null) {
              return true;
            }
            final left = crossword.characters[location.left];
            if (left != null && left.downWord != null) return true;
            final right = crossword.characters[location.right];
            if (right != null && right.downWord != null) return true;
            final up = crossword.characters[location.up];
            if (up != null && up.acrossWord != null) return true;
            final down = crossword.characters[location.down];
            if (down != null && down.acrossWord != null) return true;
            return false;
          }),
        )
        .forEach((location, character) {
          b.locationsToTry.addAll({
            location: switch ((character.acrossWord, character.downWord)) {
              (null, null) => throw StateError(
                'Character is not part of a word',
              ),
              (null, _) => Direction.across,
              (_, null) => Direction.down,
              (_, _) => throw StateError('Character is part of two words'),
            },
          });
        });
  }
});

WorkQueue remove(Location location) => rebuild(
  (b) => b
    ..locationsToTry.remove(location)
    ..badLocations.add(location),
);

/// Update the work queue from a crossword derived from the current crossword
/// that this work queue is built from.
WorkQueue updateFrom(final Crossword crossword) =>
    WorkQueue.from(
      crossword: crossword,
      candidateWords: candidateWords,
      startLocation: locationsToTry.isNotEmpty
          ? locationsToTry.keys.first
          : Location.at(0, 0),
    ).rebuild(
      (b) => b
        ..badLocations.addAll(badLocations)
        ..locationsToTry.removeWhere(
          (location, _) => badLocations.contains(location),
```

```
          ),
      );

  /// Factory constructor for [WorkQueue]
  factory WorkQueue([void Function(WorkQueueBuilder)? updates]) = _$WorkQueue;

  WorkQueue._();
}                                                       // To here.

/// Construct the serialization/deserialization code for the data model.
@SerializersFor([
  Location,
  Crossword,
  CrosswordWord,
  CrosswordCharacter,
  WorkQueue,                                            // Add this line
])
final Serializers serializers = _$serializers;
```

2. 如果新增內容後，檔案中仍有紅色波浪線，請確認 `build_runner` 是否仍在執行。如果沒有，請執行 `dart run build_runner watch -d` 指令。

您即將在程式碼中導入記錄功能，顯示建立各種大小的填字遊戲所需的時間。如果時間長度能以某種格式清楚顯示，那就太棒了。幸好，我們可以透過擴充方法新增所需的方法。

3. 按照下列方式編輯 `utils.dart` 檔案：

lib/utils.dart

```
import 'dart:math';

import 'package:built_collection/built_collection.dart';

/// A [Random] instance for generating random numbers.
final _random = Random();

/// An extension on [BuiltSet] that adds a method to get a random element.
extension RandomElements<E> on BuiltSet<E> {
  E randomElement() {
    return elementAt(_random.nextInt(length));
  }
}
                                                              // Add from here
/// An extension on [Duration] that adds a method to format the duration.
extension DurationFormat on Duration {
  /// A human-readable string representation of the duration.
  /// This format is tuned for durations in the seconds to days range.
  String get formatted {
    final hours = inHours.remainder(24).toString().padLeft(2, '0');
    final minutes = inMinutes.remainder(60).toString().padLeft(2, '0');
    final seconds = inSeconds.remainder(60).toString().padLeft(2, '0');
    return switch ((inDays, inHours, inMinutes, inSeconds)) {
      (0, 0, 0, _) => '${inSeconds}s',
      (0, 0, _, _) => '$inMinutes:$seconds',
      (0, _, _, _) => '$inHours:$minutes:$seconds',
      _ => '$inDays days, $hours:$minutes:$seconds',
    };
  }
}                                                             // To here.
```

這個擴充方法會利用記錄上的切換運算式和模式比對，選取適當的方式來顯示不同時間長度 (從秒到天)。如要進一步瞭解這種程式碼樣式，請參閱「深入瞭解 Dart 的模式和記錄」程式碼研究室。

4. 如要整合這項新功能，請替換 `isolates.dart` 檔案，重新定義 `exploreCrosswordSolutions` 函式的定義方式，如下所示：

lib/isolates.dart

```dart
import 'package:built_collection/built_collection.dart';
import 'package:characters/characters.dart';
import 'package:flutter/foundation.dart';

import 'model.dart';
import 'utils.dart';

Stream<Crossword> exploreCrosswordSolutions({
  required Crossword crossword,
  required BuiltSet<String> wordList,
}) async* {
  final start = DateTime.now();
  var workQueue = WorkQueue.from(
    crossword: crossword,
    candidateWords: wordList,
    startLocation: Location.at(0, 0),
  );
  while (!workQueue.isCompleted) {
    final location = workQueue.locationsToTry.keys.toBuiltSet().randomElement();
    try {
      final crossword = await compute(((WorkQueue, Location) workMessage) {
        final (workQueue, location) = workMessage;
        final direction = workQueue.locationsToTry[location]!;
        final target = workQueue.crossword.characters[location];
        if (target == null) {
          return workQueue.crossword.addWord(
            direction: direction,
            location: location,
            word: workQueue.candidateWords.randomElement(),
          );
        }
        var words = workQueue.candidateWords.toBuiltList().rebuild(
          (b) => b
            ..where((b) => b.characters.contains(target.character))
            ..shuffle(),
        );
        int tryCount = 0;
        for (final word in words) {
          tryCount++;
          for (final (index, character) in word.characters.indexed) {
            if (character != target.character) continue;

            final candidate = workQueue.crossword.addWord(
              location: switch (direction) {
                Direction.across => location.leftOffset(index),
                Direction.down => location.upOffset(index),
              },
              word: word,
              direction: direction,
            );
            if (candidate != null) {
              return candidate;
```

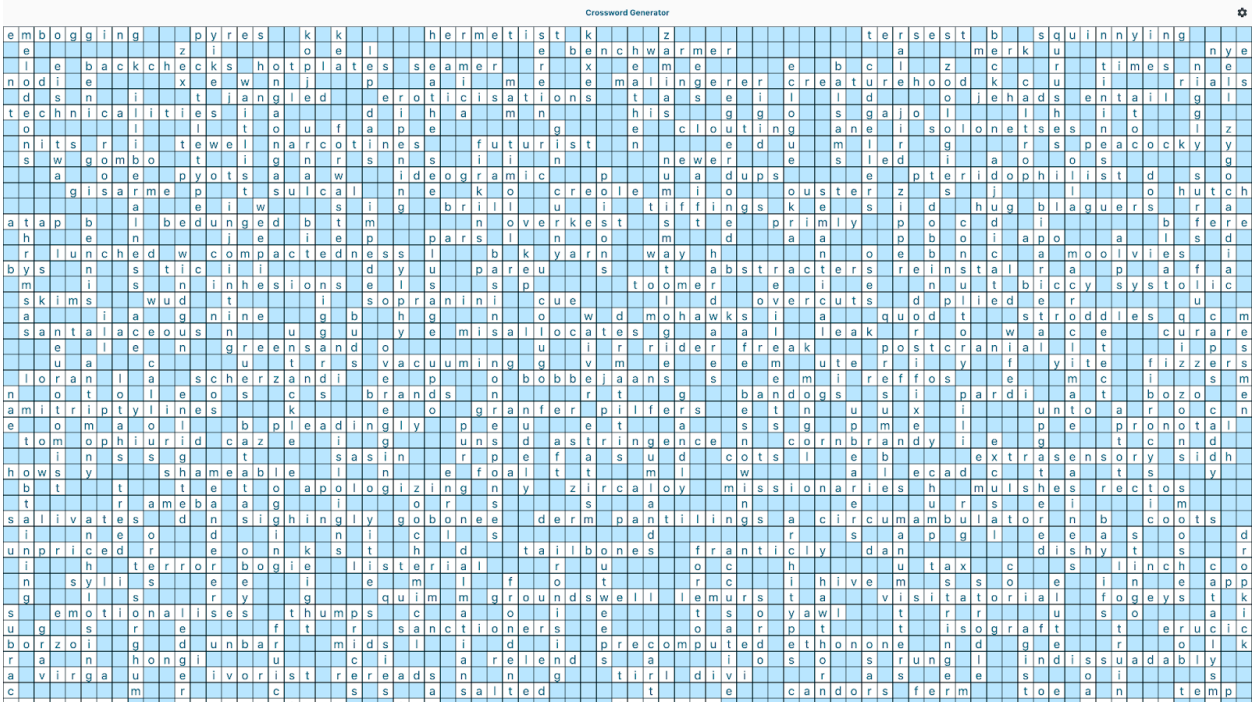
```
            }
          }
          if (tryCount > 1000) {
            break;
          }
        }
      }, (workQueue, location));
      if (crossword != null) {
        workQueue = workQueue.updateFrom(crossword);
        yield crossword;
      } else {
        workQueue = workQueue.remove(location);
      }
    } catch (e) {
      debugPrint('Error running isolate: $e');
    }
  }
  debugPrint(
    '${crossword.width} x ${crossword.height} Crossword generated in '
    '${DateTime.now().difference(start).formatted}',
  );
}
```

執行這段程式碼後，應用程式外觀會完全相同，但尋找完成的填字遊戲所需的時間不同。以下是 80 x 44 的填字遊戲，生成時間為 1 分 29 秒。

檢查點：演算法運作效率

現在生成填字遊戲的速度應該會大幅提升，原因如下：

- 智慧字詞刊登位置指定目標交集點
- 投放位置失敗時有效回溯
- 工作佇列管理，避免重複搜尋



當然，顯而易見的問題是，我們能否加快速度？當然可以。

步驟 7 · 約 5 分鐘

7. 途徑統計資料

為何要新增統計資料？

如要快速製作內容，瞭解現況很有幫助。統計資料可協助你監控進度，即時瞭解演算法的成效。您可以瞭解演算法花費時間的位置，找出瓶頸。您可以根據這些資料做出明智的決策，選擇合適的最佳化方法，進而提升成效。

您需要從 WorkQueue 擷取要顯示的資訊，並在 UI 中顯示。首先，您可以定義新的模型類別，其中包含要顯示的資訊。

如要開始，請按照下列步驟操作：

1. 按照下列方式編輯 `model.dart` 檔案，新增 `DisplayInfo` 類別：

lib/model.dart

```
import 'package:built_collection/built_collection.dart';
import 'package:built_value/built_value.dart';
import 'package:built_value/serializer.dart';
import 'package:characters/characters.dart';
import 'package:intl/intl.dart';                          // Add this import

part 'model.g.dart';

/// A location in a crossword.
abstract class Location implements Built<Location, LocationBuilder> {
```

2. 在檔案結尾進行下列變更，新增 `DisplayInfo` 類別：

lib/model.dart

```
  /// Factory constructor for [WorkQueue]
  factory WorkQueue([void Function(WorkQueueBuilder)? updates]) = _$WorkQueue;

  WorkQueue._();
}
                                                          // Add from here.
/// Display information for the current state of the crossword solve.
abstract class DisplayInfo implements Built<DisplayInfo, DisplayInfoBuilder> {
  static Serializer<DisplayInfo> get serializer => _$displayInfoSerializer;

  /// The number of words in the grid.
  String get wordsInGridCount;

  /// The number of candidate words.
  String get candidateWordsCount;

  /// The number of locations to explore.
  String get locationsToExploreCount;

  /// The number of known bad locations.
  String get knownBadLocationsCount;

  /// The percentage of the grid filled.
  String get gridFilledPercentage;

  /// Construct a [DisplayInfo] instance from a [WorkQueue].
  factory DisplayInfo.from({required WorkQueue workQueue}) {
    final gridFilled =
        (workQueue.crossword.characters.length /
        (workQueue.crossword.width * workQueue.crossword.height));
    final fmt = NumberFormat.decimalPattern();

    return DisplayInfo(
      (b) => b
        ..wordsInGridCount = fmt.format(workQueue.crossword.words.length)
        ..candidateWordsCount = fmt.format(workQueue.candidateWords.length)
        ..locationsToExploreCount = fmt.format(workQueue.locationsToTry.length)
        ..knownBadLocationsCount = fmt.format(workQueue.badLocations.length)
        ..gridFilledPercentage = '${(gridFilled * 100).toStringAsFixed(2)}%',
    );
  }

  /// An empty [DisplayInfo] instance.
  static DisplayInfo get empty => DisplayInfo(
    (b) => b
      ..wordsInGridCount = '0'
      ..candidateWordsCount = '0'
      ..locationsToExploreCount = '0'
      ..knownBadLocationsCount = '0'
      ..gridFilledPercentage = '0%',
  );
```

```
  factory DisplayInfo([void Function(DisplayInfoBuilder)? updates]) =
      _$DisplayInfo;
  DisplayInfo._();
}                                                          // To here.

/// Construct the serialization/deserialization code for the data model.
@SerializersFor([
  Location,
  Crossword,
  CrosswordWord,
  CrosswordCharacter,
  WorkQueue,
  DisplayInfo,                                             // Add this line.
])
final Serializers serializers = _$serializers;
```

3. 修改 `isolates.dart` 檔案，公開 `WorkQueue` 模型，如下所示：

lib/isolates.dart

```dart
import 'package:built_collection/built_collection.dart';
import 'package:characters/characters.dart';
import 'package:flutter/foundation.dart';

import 'model.dart';
import 'utils.dart';

Stream<WorkQueue> exploreCrosswordSolutions({              // Modify this line
  required Crossword crossword,
  required BuiltSet<String> wordList,
}) async* {
  final start = DateTime.now();
  var workQueue = WorkQueue.from(
    crossword: crossword,
    candidateWords: wordList,
    startLocation: Location.at(0, 0),
  );
  while (!workQueue.isCompleted) {
    final location = workQueue.locationsToTry.keys.toBuiltSet().randomElement();
    try {
      final crossword = await compute(((WorkQueue, Location) workMessage) {
        final (workQueue, location) = workMessage;
        final direction = workQueue.locationsToTry[location]!;
        final target = workQueue.crossword.characters[location];
        if (target == null) {
          return workQueue.crossword.addWord(
            direction: direction,
            location: location,
            word: workQueue.candidateWords.randomElement(),
          );
        }
        var words = workQueue.candidateWords.toBuiltList().rebuild(
          (b) => b
            ..where((b) => b.characters.contains(target.character))
            ..shuffle(),
        );
        int tryCount = 0;
        for (final word in words) {
          tryCount++;
          for (final (index, character) in word.characters.indexed) {
            if (character != target.character) continue;

            final candidate = workQueue.crossword.addWord(
              location: switch (direction) {
                Direction.across => location.leftOffset(index),
                Direction.down => location.upOffset(index),
              },
              word: word,
              direction: direction,
            );
            if (candidate != null) {
              return candidate;
```

```
            }
          }
          if (tryCount > 1000) {
            break;
          }
        }
      }, (workQueue, location));
      if (crossword != null) {
        workQueue = workQueue.updateFrom(crossword);       // Drop the yield crossword;
      } else {
        workQueue = workQueue.remove(location);
      }
      yield workQueue;                                     // Add this line.
    } catch (e) {
      debugPrint('Error running isolate: $e');
    }
  }
  debugPrint(
    '${crossword.width} x ${crossword.height} Crossword generated in '
    '${DateTime.now().difference(start).formatted}',
  );
}
```

背景隔離區現在會公開工作佇列，因此現在的問題是，如何以及從何處從這個資料來源衍生統計資料。

4. 將舊的填字遊戲供應商換成工作佇列供應商，然後新增更多供應商，從工作佇列供應商的串流衍生資訊：

lib/providers.dart

```dart
import 'dart:convert';
import 'dart:math';                                          // Add this import

import 'package:built_collection/built_collection.dart';
import 'package:flutter/foundation.dart';
import 'package:flutter/services.dart';
import 'package:riverpod/riverpod.dart';
import 'package:riverpod_annotation/riverpod_annotation.dart';

import 'isolates.dart';
import 'model.dart' as model;

part 'providers.g.dart';

/// A provider for the wordlist to use when generating the crossword.
@riverpod
Future<BuiltSet<String>> wordList(Ref ref) async {
  // This codebase requires that all words consist of lowercase characters
  // in the range 'a'-'z'. Words containing uppercase letters will be
  // lowercased, and words containing runes outside this range will
  // be removed.

  final re = RegExp(r'^[a-z]+$');
  final words = await rootBundle.loadString('assets/words.txt');
  return const LineSplitter()
      .convert(words)
      .toBuiltSet()
      .rebuild(
        (b) => b
          ..map((word) => word.toLowerCase().trim())
          ..where((word) => word.length > 2)
          ..where((word) => re.hasMatch(word)),
      );
}

/// An enumeration for different sizes of [model.Crossword]s.
enum CrosswordSize {
  small(width: 20, height: 11),
  medium(width: 40, height: 22),
  large(width: 80, height: 44),
  xlarge(width: 160, height: 88),
  xxlarge(width: 500, height: 500);

  const CrosswordSize({required this.width, required this.height});

  final int width;
  final int height;
  String get label => '$width x $height';
}

/// A provider that holds the current size of the crossword to generate.
@Riverpod(keepAlive: true)
```

```
class Size extends _$Size {
  var _size = CrosswordSize.medium;

  @override
  CrosswordSize build() => _size;

  void setSize(CrosswordSize size) {
    _size = size;
    ref.invalidateSelf();
  }
}

@riverpod
Stream<model.WorkQueue> workQueue(Ref ref) async* {        // Modify this provider
  final size = ref.watch(sizeProvider);
  final wordListAsync = ref.watch(wordListProvider);
  final emptyCrossword = model.Crossword.crossword(
    width: size.width,
    height: size.height,
  );
  final emptyWorkQueue = model.WorkQueue.from(
    crossword: emptyCrossword,
    candidateWords: BuiltSet<String>(),
    startLocation: model.Location.at(0, 0),
  );

  ref.read(startTimeProvider.notifier).start();
  ref.read(endTimeProvider.notifier).clear();

  yield* wordListAsync.when(
    data: (wordList) => exploreCrosswordSolutions(
      crossword: emptyCrossword,
      wordList: wordList,
    ),
    error: (error, stackTrace) async* {
      debugPrint('Error loading word list: $error');
      yield emptyWorkQueue;
    },
    loading: () async* {
      yield emptyWorkQueue;
    },
  );

  ref.read(endTimeProvider.notifier).end();
}                                                          // To here.

@Riverpod(keepAlive: true)                                 // Add from here to end of file
class StartTime extends _$StartTime {
  @override
  DateTime? build() => _start;

  DateTime? _start;
```

```
  void start() {
    _start = DateTime.now();
    ref.invalidateSelf();
  }
}

@Riverpod(keepAlive: true)
class EndTime extends _$EndTime {
  @override
  DateTime? build() => _end;

  DateTime? _end;

  void clear() {
    _end = null;
    ref.invalidateSelf();
  }

  void end() {
    _end = DateTime.now();
    ref.invalidateSelf();
  }
}

const _estimatedTotalCoverage = 0.54;

@riverpod
Duration expectedRemainingTime(Ref ref) {
  final startTime = ref.watch(startTimeProvider);
  final endTime = ref.watch(endTimeProvider);
  final workQueueAsync = ref.watch(workQueueProvider);

  return workQueueAsync.when(
    data: (workQueue) {
      if (startTime == null || endTime != null || workQueue.isCompleted) {
        return Duration.zero;
      }
      try {
        final soFar = DateTime.now().difference(startTime);
        final completedPercentage = min(
          0.99,
          (workQueue.crossword.characters.length /
              (workQueue.crossword.width * workQueue.crossword.height) /
              _estimatedTotalCoverage),
        );
        final expectedTotal = soFar.inSeconds / completedPercentage;
        final expectedRemaining = expectedTotal - soFar.inSeconds;
        return Duration(seconds: expectedRemaining.toInt());
      } catch (e) {
        return Duration.zero;
      }
```

```
    },
    error: (error, stackTrace) => Duration.zero,
    loading: () => Duration.zero,
  );
}

/// A provider that holds whether to display info.
@Riverpod(keepAlive: true)
class ShowDisplayInfo extends _$ShowDisplayInfo {
  var _display = true;

  @override
  bool build() => _display;

  void toggle() {
    _display = !_display;
    ref.invalidateSelf();
  }
}

/// A provider that summarise the DisplayInfo from a [model.WorkQueue].
@riverpod
class DisplayInfo extends _$DisplayInfo {
  @override
  model.DisplayInfo build() => ref
      .watch(workQueueProvider)
      .when(
        data: (workQueue) => model.DisplayInfo.from(workQueue: workQueue),
        error: (error, stackTrace) => model.DisplayInfo.empty,
        loading: () => model.DisplayInfo.empty,
      );
}
```

新供應商包括全域狀態 (資訊顯示是否應疊加在填字遊戲網格上)，以及衍生資料 (例如填字遊戲生成作業的執行時間)。此外，部分狀態的監聽器是暫時性的，這也讓情況更加複雜。如果隱藏資訊顯示畫面，系統就不會監聽填字遊戲計算的開始和結束時間，但如果要在顯示資訊時準確計算，這些時間就必須保留在記憶體中。在這種情況下，`Riverpod` 屬性的 `keepAlive` 參數非常實用。

顯示資訊時，會出現輕微的皺紋。我們希望能夠顯示經過的執行時間，但這裡沒有任何內容可強制持續更新經過的時間。回到「在 Flutter 中建構新一代 UI」程式碼研究室，您會發現一個實用的小工具，正好符合這項需求。

5. 在 `lib/widgets` 目錄中建立 `ticker_builder.dart` 檔案，然後加入以下內容：

lib/widgets/ticker_builder.dart

```
import 'package:flutter/material.dart';
import 'package:flutter/scheduler.dart';

/// A Builder widget that invokes its [builder] function on every animation frame.
class TickerBuilder extends StatefulWidget {
  const TickerBuilder({super.key, required this.builder});
  final Widget Function(BuildContext context) builder;
  @override
  State<TickerBuilder> createState() => _TickerBuilderState();
}

class _TickerBuilderState extends State<TickerBuilder>
    with SingleTickerProviderStateMixin {
  late final Ticker _ticker;

  @override
  void initState() {
    super.initState();
    _ticker = createTicker(_handleTick)..start();
  }

  @override
  void dispose() {
    _ticker.dispose();
    super.dispose();
  }

  void _handleTick(Duration elapsed) {
    setState(() {
      // Force a rebuild without changing the widget tree.
    });
  }

  @override
  Widget build(BuildContext context) => widget.builder.call(context);
}
```

這個小工具是鐵鎚。並在每個影格上重建內容。一般來說，這種做法並不建議，但相較於搜尋填字遊戲的運算負載，每影格重新繪製經過時間的運算負載可能會消失在雜訊中。如要善用這項新衍生資訊，現在請建立新的小工具。

6. 在 `lib/widgets` 目錄中建立 `crossword_info_widget.dart` 檔案，然後加入以下內容：

lib/widgets/crossword_info_widget.dart

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../providers.dart';
import '../utils.dart';
import 'ticker_builder.dart';

class CrosswordInfoWidget extends ConsumerWidget {
  const CrosswordInfoWidget({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final size = ref.watch(sizeProvider);
    final displayInfo = ref.watch(displayInfoProvider);
    final startTime = ref.watch(startTimeProvider);
    final endTime = ref.watch(endTimeProvider);
    final remaining = ref.watch(expectedRemainingTimeProvider);

    return Align(
      alignment: Alignment.bottomRight,
      child: Padding(
        padding: const EdgeInsets.only(right: 32.0, bottom: 32.0),
        child: ClipRRect(
          borderRadius: BorderRadius.circular(8),
          child: ColoredBox(
            color: Theme.of(context).colorScheme.onPrimary.withAlpha(230),
            child: Padding(
              padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 8),
              child: DefaultTextStyle(
                style: TextStyle(
                  fontSize: 16,
                  color: Theme.of(context).colorScheme.primary,
                ),
                child: Column(
                  mainAxisSize: MainAxisSize.min,
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    _CrosswordInfoRichText(
                      label: 'Grid Size',
                      value: '${size.width} x ${size.height}',
                    ),
                    _CrosswordInfoRichText(
                      label: 'Words in grid',
                      value: displayInfo.wordsInGridCount,
                    ),
                    _CrosswordInfoRichText(
                      label: 'Candidate words',
                      value: displayInfo.candidateWordsCount,
                    ),
                    _CrosswordInfoRichText(
                      label: 'Locations to explore',
                      value: displayInfo.locationsToExploreCount,
```

```
                    ),
                    _CrosswordInfoRichText(
                      label: 'Known bad locations',
                      value: displayInfo.knownBadLocationsCount,
                    ),
                    _CrosswordInfoRichText(
                      label: 'Grid filled',
                      value: displayInfo.gridFilledPercentage,
                    ),
                    switch ((startTime, endTime)) {
                      (null, _) => _CrosswordInfoRichText(
                        label: 'Time elapsed',
                        value: 'Not started yet',
                      ),
                      (DateTime start, null) => TickerBuilder(
                        builder: (context) => _CrosswordInfoRichText(
                          label: 'Time elapsed',
                          value: DateTime.now().difference(start).formatted,
                        ),
                      ),
                      (DateTime start, DateTime end) => _CrosswordInfoRichText(
                        label: 'Completed in',
                        value: end.difference(start).formatted,
                      ),
                    },
                    if (startTime != null && endTime == null)
                      _CrosswordInfoRichText(
                        label: 'Est. remaining',
                        value: remaining.formatted,
                      ),
                  ],
                ),
              ),
            ),
          ),
        ),
      ),
    );
  }
}

class _CrosswordInfoRichText extends StatelessWidget {
  final String label;
  final String value;

  const _CrosswordInfoRichText({required this.label, required this.value});

  @override
  Widget build(BuildContext context) => RichText(
    text: TextSpan(
      children: [
        TextSpan(text: '$label ', style: DefaultTextStyle.of(context).style),
```

```
        TextSpan(
          text: value,
          style: DefaultTextStyle.of(
            context,
          ).style.copyWith(fontWeight: FontWeight.bold),
        ),
      ],
    ),
  );
}
```

這個小工具是 Riverpod 供應器強大的最佳範例。當五個供應商中有任何一個更新時，這個小工具就會標示為重建。這個步驟的最後一項必要變更，是將這個新小工具整合到 UI 中。

7. 按照下列方式編輯 `crossword_generator_app.dart` 檔案：

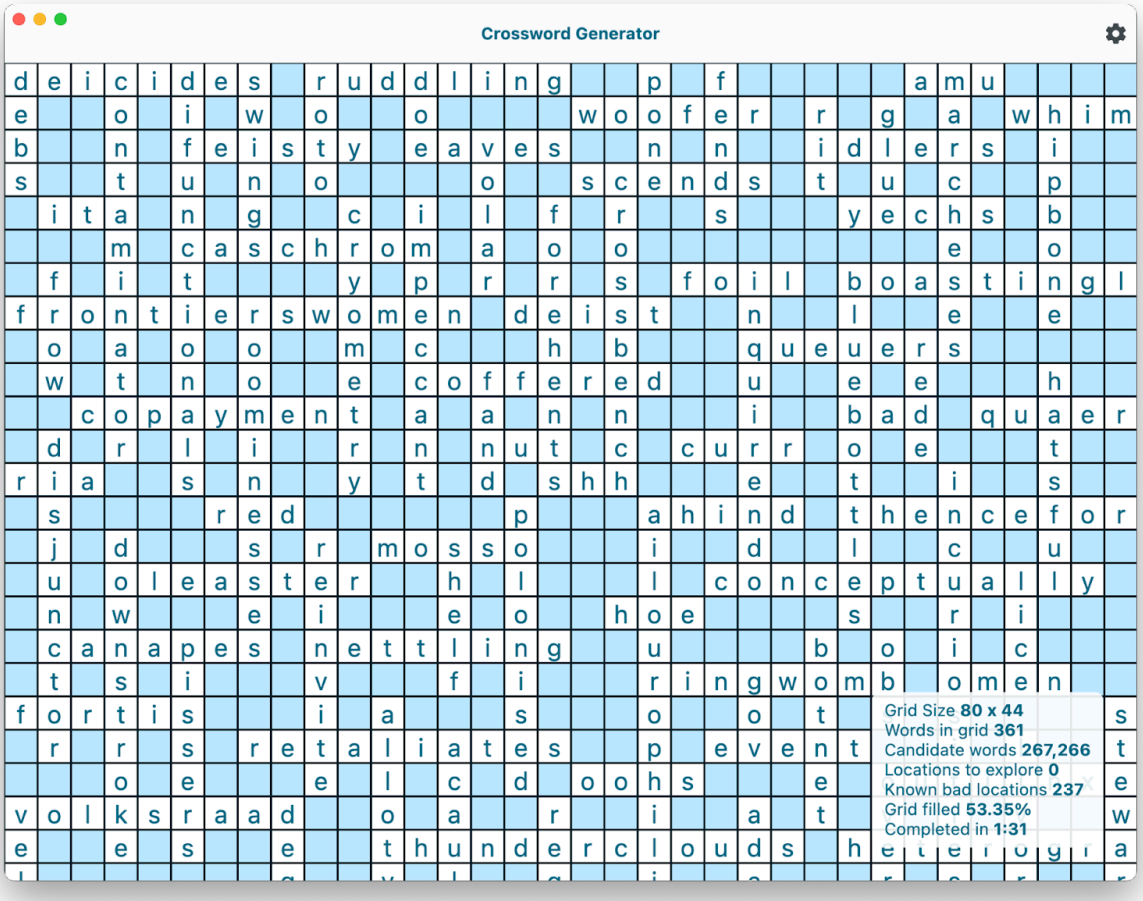
lib/widgets/crossword_generator_app.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../providers.dart';
import 'crossword_info_widget.dart';                        // Add this import
import 'crossword_widget.dart';

class CrosswordGeneratorApp extends StatelessWidget {
  const CrosswordGeneratorApp({super.key});

  @override
  Widget build(BuildContext context) {
    return _EagerInitialization(
      child: Scaffold(
        appBar: AppBar(
          actions: [_CrosswordGeneratorMenu()],
          titleTextStyle: TextStyle(
            color: Theme.of(context).colorScheme.primary,
            fontSize: 16,
            fontWeight: FontWeight.bold,
          ),
          title: Text('Crossword Generator'),
        ),
        body: SafeArea(
          child: Consumer(                                  // Modify from here
            builder: (context, ref, child) {
              return Stack(
                children: [
                  Positioned.fill(child: CrosswordWidget()),
                  if (ref.watch(showDisplayInfoProvider)) CrosswordInfoWidget(),
                ],
              );
            },
          ),                                                // To here.
        ),
      ),
    );
  }
}

class _EagerInitialization extends ConsumerWidget {
  const _EagerInitialization({required this.child});
  final Widget child;

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    ref.watch(wordListProvider);
    return child;
  }
}

class _CrosswordGeneratorMenu extends ConsumerWidget {
```

```
  @override
  Widget build(BuildContext context, WidgetRef ref) => MenuAnchor(
    menuChildren: [
      for (final entry in CrosswordSize.values)
        MenuItemButton(
          onPressed: () => ref.read(sizeProvider.notifier).setSize(entry),
          leadingIcon: entry == ref.watch(sizeProvider)
              ? Icon(Icons.radio_button_checked_outlined)
              : Icon(Icons.radio_button_unchecked_outlined),
          child: Text(entry.label),
        ),
      MenuItemButton(                                        // Add from here
        leadingIcon: ref.watch(showDisplayInfoProvider)
            ? Icon(Icons.check_box_outlined)
            : Icon(Icons.check_box_outline_blank_outlined),
        onPressed: () => ref.read(showDisplayInfoProvider.notifier).toggle(),
        child: Text('Display Info'),
      ),                                                     // To here.
    ],
    builder: (context, controller, child) => IconButton(
      onPressed: () => controller.open(),
      icon: Icon(Icons.settings),
    ),
  );
}
```

這兩項變更分別代表整合供應商的不同方法。在 `CrosswordGeneratorApp` 的 `build` 方法中，您導入了新的 `Consumer` 建構工具，用來包含資訊顯示或隱藏時強制重建的區域。另一方面，整個下拉式選單是單一 `ConsumerWidget`，無論是調整填字遊戲大小，還是顯示/隱藏資訊顯示畫面，都會重建選單。要採取哪種做法，一律是工程上的取捨，也就是簡化與重新計算重建小工具樹狀結構的版面配置成本之間的取捨。

現在執行應用程式，使用者就能進一步瞭解填字遊戲的生成進度。不過，在生成字謎快結束時，我們發現數字會變動，但字元格的變化很小。



取得更多資訊，瞭解發生了什麼事以及原因，會很有幫助。

步驟 8 · 約 5 分鐘

8. 使用執行緒平行處理

效能降低的原因

隨著填字遊戲接近完成，演算法會放慢速度，因為可用的字詞放置選項較少。演算法會嘗試許多無效的組合。單一執行緒處理無法有效探索多個選項

演算法的視覺化呈現

如要瞭解為何最後會變慢，可視需要將演算法的運作方式視覺化。其中一個重要部分是 `locationsToTry` 中出色的 `WorkQueue`。TableView 提供實用的方式來調查這個問題。我們可以根據儲存格是否位於 `locationsToTry` 中，變更儲存格顏色。

如要開始，請按照下列步驟操作：

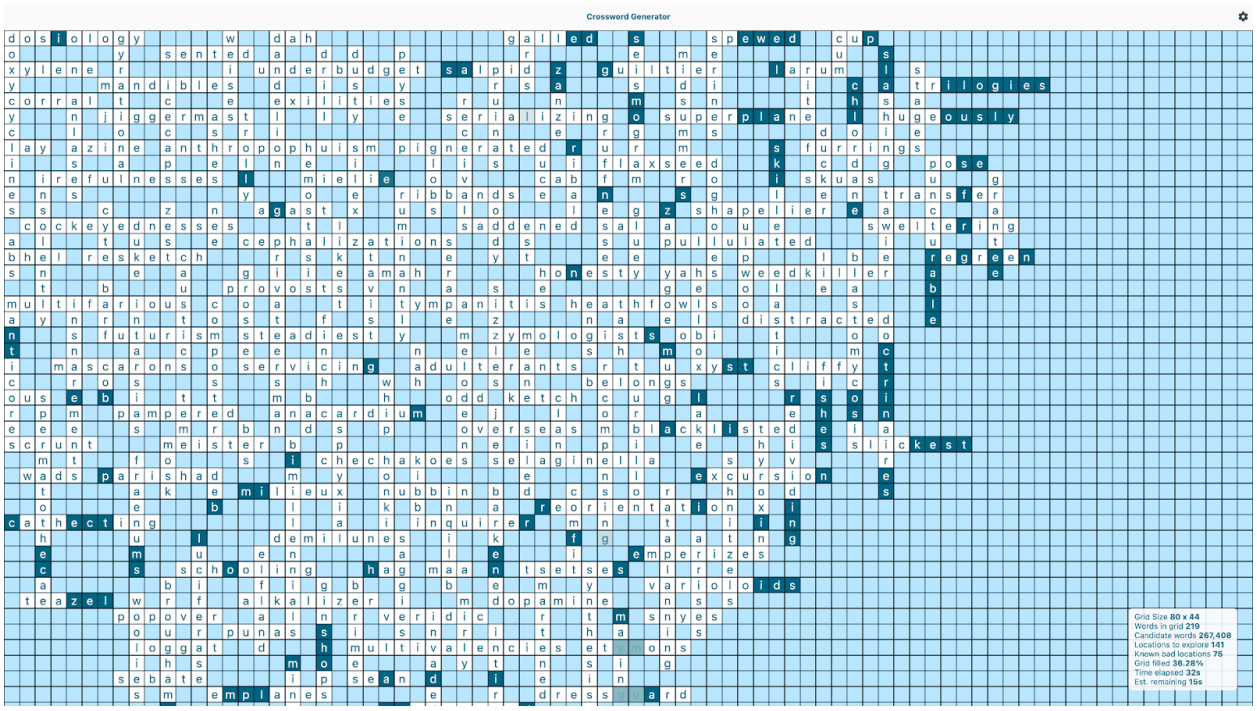
1. 按照下列方式修改 `crossword_widget.dart` 檔案：

lib/widgets/crossword_widget.dart

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:two_dimensional_scrollables/two_dimensional_scrollables.dart';

import '../model.dart';
import '../providers.dart';

class CrosswordWidget extends ConsumerWidget {
  const CrosswordWidget({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final size = ref.watch(sizeProvider);
    return TableView.builder(
      diagonalDragBehavior: DiagonalDragBehavior.free,
      cellBuilder: _buildCell,
      columnCount: size.width,
      columnBuilder: (index) => _buildSpan(context, index),
      rowCount: size.height,
      rowBuilder: (index) => _buildSpan(context, index),
    );
  }

  TableViewCell _buildCell(BuildContext context, TableVicinity vicinity) {
    final location = Location.at(vicinity.column, vicinity.row);

    return TableViewCell(
      child: Consumer(
        builder: (context, ref, _) {
          final character = ref.watch(
            workQueueProvider.select(
              (workQueueAsync) => workQueueAsync.when(
                data: (workQueue) => workQueue.crossword.characters[location],
                error: (error, stackTrace) => null,
                loading: () => null,
              ),
            ),
          );

          final explorationCell = ref.watch(               // Add from here
            workQueueProvider.select(
              (workQueueAsync) => workQueueAsync.when(
                data: (workQueue) =>
                    workQueue.locationsToTry.keys.contains(location),
                error: (error, stackTrace) => false,
                loading: () => false,
              ),
            ),
          );                                               // To here.

          if (character != null) {                         // Modify from here
            return AnimatedContainer(
```

```
              duration: Durations.extralong1,
              curve: Curves.easeInOut,
              color: explorationCell
                  ? Theme.of(context).colorScheme.primary
                  : Theme.of(context).colorScheme.onPrimary,
              child: Center(
                child: AnimatedDefaultTextStyle(
                  duration: Durations.extralong1,
                  curve: Curves.easeInOut,
                  style: TextStyle(
                    fontSize: 24,
                    color: explorationCell
                        ? Theme.of(context).colorScheme.onPrimary
                        : Theme.of(context).colorScheme.primary,
                  ),
                  child: Text(character.character),
                ),                                          // To here.
              ),
            );
          }

          return ColoredBox(
            color: Theme.of(context).colorScheme.primaryContainer,
          );
        },
      ),
    );
  }

  TableSpan _buildSpan(BuildContext context, int index) {
    return TableSpan(
      extent: FixedTableSpanExtent(32),
      foregroundDecoration: TableSpanDecoration(
        border: TableSpanBorder(
          leading: BorderSide(
            color: Theme.of(context).colorScheme.onPrimaryContainer,
          ),
          trailing: BorderSide(
            color: Theme.of(context).colorScheme.onPrimaryContainer,
          ),
        ),
      ),
    );
  }
}
```

執行這段程式碼後，您會看到演算法尚未調查的待處理位置。



有趣的是，在填寫填字遊戲的過程中，會發現許多調查點不會產生任何有用的結果。這裡有兩個選項：一是填入一定比例的填字遊戲儲存格後，就停止調查；二是同時調查多個感興趣的點。第二條路聽起來比較有趣，因此現在就來實作。

2. 編輯 `isolates.dart` 檔案。這幾乎是程式碼的完整重寫，可將一個背景隔離區中計算的內容分割成 N 個背景隔離區的集區。

lib/isolates.dart

```
import 'package:built_collection/built_collection.dart';
import 'package:characters/characters.dart';
import 'package:flutter/foundation.dart';

import 'model.dart';
import 'utils.dart';

Stream<WorkQueue> exploreCrosswordSolutions({
  required Crossword crossword,
  required BuiltSet<String> wordList,
  required int maxWorkerCount,
}) async* {
  final start = DateTime.now();
  var workQueue = WorkQueue.from(
    crossword: crossword,
    candidateWords: wordList,
    startLocation: Location.at(0, 0),
  );
  while (!workQueue.isCompleted) {
    try {
      workQueue = await compute(_generate, (workQueue, maxWorkerCount));
      yield workQueue;
    } catch (e) {
      debugPrint('Error running isolate: $e');
    }
  }

  debugPrint(
    'Generated ${workQueue.crossword.width} x '
    '${workQueue.crossword.height} crossword in '
    '${DateTime.now().difference(start).formatted} '
    'with $maxWorkerCount workers.',
  );
}

Future<WorkQueue> _generate((WorkQueue, int) workMessage) async {
  var (workQueue, maxWorkerCount) = workMessage;
  final candidateGeneratorFutures = <Future<(Location, Direction, String?)>>[];
  final locations = workQueue.locationsToTry.keys.toBuiltList().rebuild(
    (b) => b
      ..shuffle()
      ..take(maxWorkerCount),
  );

  for (final location in locations) {
    final direction = workQueue.locationsToTry[location]!;

    candidateGeneratorFutures.add(
      compute(_generateCandidate, (
        workQueue.crossword,
        workQueue.candidateWords,
        location,
```

```
        direction,
      )),
    );
  }

  try {
    final results = await candidateGeneratorFutures.wait;
    var crossword = workQueue.crossword;
    for (final (location, direction, word) in results) {
      if (word != null) {
        final candidate = crossword.addWord(
          location: location,
          word: word,
          direction: direction,
        );
        if (candidate != null) {
          crossword = candidate;
        }
      } else {
        workQueue = workQueue.remove(location);
      }
    }

    workQueue = workQueue.updateFrom(crossword);
  } catch (e) {
    debugPrint('$e');
  }

  return workQueue;
}

(Location, Direction, String?) _generateCandidate(
  (Crossword, BuiltSet<String>, Location, Direction) searchDetailMessage,
) {
  final (crossword, candidateWords, location, direction) = searchDetailMessage;

  final target = crossword.characters[location];
  if (target == null) {
    return (location, direction, candidateWords.randomElement());
  }

  // Filter down the candidate word list to those that contain the letter
  // at the current location
  final words = candidateWords.toBuiltList().rebuild(
    (b) => b
      ..where((b) => b.characters.contains(target.character))
      ..shuffle(),
  );
  int tryCount = 0;
  final start = DateTime.now();
  for (final word in words) {
    tryCount++;
```

```
    for (final (index, character) in word.characters.indexed) {
      if (character != target.character) continue;

      final candidate = crossword.addWord(
        location: switch (direction) {
          Direction.across => location.leftOffset(index),
          Direction.down => location.upOffset(index),
        },
        word: word,
        direction: direction,
      );
      if (candidate != null) {
        return switch (direction) {
          Direction.across => (location.leftOffset(index), direction, word),
          Direction.down => (location.upOffset(index), direction, word),
        };
      }
      final deltaTime = DateTime.now().difference(start);
      if (tryCount >= 1000 || deltaTime > Duration(seconds: 10)) {
        return (location, direction, null);
      }
    }
  }

  return (location, direction, null);
}
```

瞭解多重隔離架構

由於核心業務邏輯並未變更，因此您應該很熟悉大部分的程式碼。現在有兩層 `compute` 呼叫，第一層負責將個別位置分配給 N 個工作人員隔離區進行搜尋，然後在所有 N 個工作人員隔離區完成作業後，重新合併結果。第二層包含 N 個工作站隔離區。如要調整 N 以獲得最佳效能，取決於電腦和相關資料。格線越大，可同時作業的員工就越多，且不會互相干擾。

值得注意的一點是，請留意這段程式碼現在如何處理閉包擷取不應擷取內容的問題。目前沒有任何關閉的商店。`_generate` 和 `_generateWorker` 函式定義為頂層函式，沒有可擷取的周圍環境。這兩個函式的引數和結果都是 Dart 記錄的形式。這是規避 `compute` 呼叫的「一個值輸入，一個值輸出」語意的方法。

您現在可以建立背景工作者集區，搜尋在格線中互鎖的字詞，組成填字遊戲，接下來要將這項功能提供給填字遊戲產生器工具的其餘部分。

3. 編輯 `providers.dart` 檔案，並按照下列方式編輯 workQueue 提供者：

lib/providers.dart

```
@riverpod
Stream<model.WorkQueue> workQueue(Ref ref) async* {
  final workers = ref.watch(workerCountProvider);          // Add this line
  final size = ref.watch(sizeProvider);
  final wordListAsync = ref.watch(wordListProvider);
  final emptyCrossword = model.Crossword.crossword(
    width: size.width,
    height: size.height,
  );
  final emptyWorkQueue = model.WorkQueue.from(
    crossword: emptyCrossword,
    candidateWords: BuiltSet<String>(),
    startLocation: model.Location.at(0, 0),
  );

  ref.read(startTimeProvider.notifier).start();
  ref.read(endTimeProvider.notifier).clear();

  yield* wordListAsync.when(
    data: (wordList) => exploreCrosswordSolutions(
      crossword: emptyCrossword,
      wordList: wordList,
      maxWorkerCount: workers.count,                       // Add this line
    ),
    error: (error, stackTrace) async* {
      debugPrint('Error loading word list: $error');
      yield emptyWorkQueue;
    },
    loading: () async* {
      yield emptyWorkQueue;
    },
  );

  ref.read(endTimeProvider.notifier).end();
}
```

4. 在檔案結尾處加入 `WorkerCount` 供應器，如下所示：

lib/providers.dart

```
/// A provider that summarise the DisplayInfo from a [model.WorkQueue].
@riverpod
class DisplayInfo extends _$DisplayInfo {
  @override
  model.DisplayInfo build() => ref
      .watch(workQueueProvider)
      .when(
        data: (workQueue) => model.DisplayInfo.from(workQueue: workQueue),
        error: (error, stackTrace) => model.DisplayInfo.empty,
        loading: () => model.DisplayInfo.empty,
      );
}

enum BackgroundWorkers {                                    // Add from here
  one(1),
  two(2),
  four(4),
  eight(8),
  sixteen(16),
  thirtyTwo(32),
  sixtyFour(64),
  oneTwentyEight(128);

  const BackgroundWorkers(this.count);

  final int count;
  String get label => count.toString();
}

/// A provider that holds the current number of background workers to use.
@Riverpod(keepAlive: true)
class WorkerCount extends _$WorkerCount {
  var _count = BackgroundWorkers.four;

  @override
  BackgroundWorkers build() => _count;

  void setCount(BackgroundWorkers count) {
    _count = count;
    ref.invalidateSelf();
  }
}                                                           // To here.
```

完成這兩項變更後，供應商層現在會公開設定背景隔離集區工作站數量上限的方法，確保隔離函式設定正確無誤。

5. 修改 `CrosswordInfoWidget`，更新 `crossword_info_widget.dart` 檔案，如下所示：

lib/widgets/crossword_info_widget.dart

```
class CrosswordInfoWidget extends ConsumerWidget {
  const CrosswordInfoWidget({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final size = ref.watch(sizeProvider);
    final displayInfo = ref.watch(displayInfoProvider);
    final workerCount = ref.watch(workerCountProvider).label;  // Add this line
    final startTime = ref.watch(startTimeProvider);
    final endTime = ref.watch(endTimeProvider);
    final remaining = ref.watch(expectedRemainingTimeProvider);

    return Align(
      alignment: Alignment.bottomRight,
      child: Padding(
        padding: const EdgeInsets.only(right: 32.0, bottom: 32.0),
        child: ClipRRect(
          borderRadius: BorderRadius.circular(8),
          child: ColoredBox(
            color: Theme.of(context).colorScheme.onPrimary.withAlpha(230),
            child: Padding(
              padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 8),
              child: DefaultTextStyle(
                style: TextStyle(
                  fontSize: 16,
                  color: Theme.of(context).colorScheme.primary,
                ),
                child: Column(
                  mainAxisSize: MainAxisSize.min,
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    _CrosswordInfoRichText(
                      label: 'Grid Size',
                      value: '${size.width} x ${size.height}',
                    ),
                    _CrosswordInfoRichText(
                      label: 'Words in grid',
                      value: displayInfo.wordsInGridCount,
                    ),
                    _CrosswordInfoRichText(
                      label: 'Candidate words',
                      value: displayInfo.candidateWordsCount,
                    ),
                    _CrosswordInfoRichText(
                      label: 'Locations to explore',
                      value: displayInfo.locationsToExploreCount,
                    ),
                    _CrosswordInfoRichText(
                      label: 'Known bad locations',
                      value: displayInfo.knownBadLocationsCount,
                    ),
                    _CrosswordInfoRichText(
```

```
                    label: 'Grid filled',
                    value: displayInfo.gridFilledPercentage,
                  ),
                  _CrosswordInfoRichText(               // Add from here
                    label: 'Max worker count',
                    value: workerCount,
                  ),                                    // To here.
                  switch ((startTime, endTime)) {
                    (null, _) => _CrosswordInfoRichText(
                      label: 'Time elapsed',
                      value: 'Not started yet',
                    ),
                    (DateTime start, null) => TickerBuilder(
                      builder: (context) => _CrosswordInfoRichText(
                        label: 'Time elapsed',
                        value: DateTime.now().difference(start).formatted,
                      ),
                    ),
                    (DateTime start, DateTime end) => _CrosswordInfoRichText(
                      label: 'Completed in',
                      value: end.difference(start).formatted,
                    ),
                  },
                  if (startTime != null && endTime == null)
                    _CrosswordInfoRichText(
                      label: 'Est. remaining',
                      value: remaining.formatted,
                    ),
                ],
              ),
            ),
          ),
        ),
      ),
    ),
  );
  }
}
```

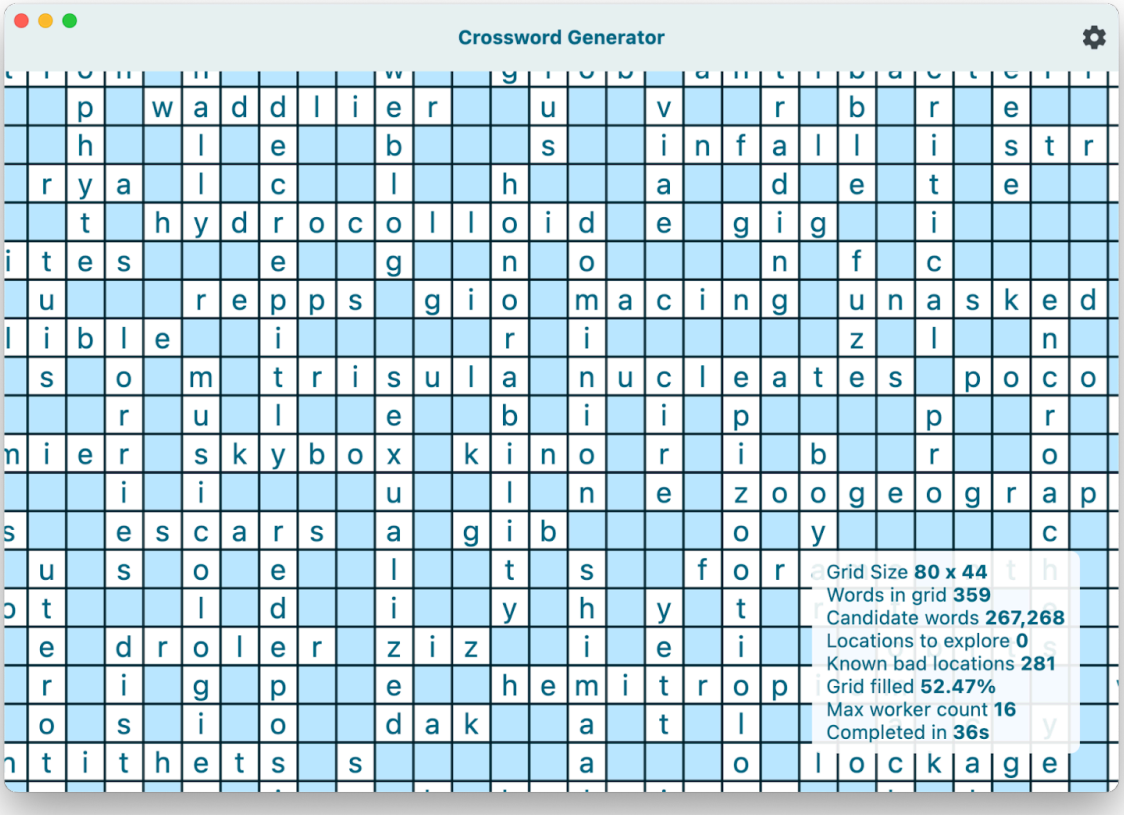
6. 修改 `crossword_generator_app.dart` 檔案，在 `_CrosswordGeneratorMenu` 小工具中新增下列部分：

lib/widgets/crossword_generator_app.dart

```
class _CrosswordGeneratorMenu extends ConsumerWidget {
  @override
  Widget build(BuildContext context, WidgetRef ref) => MenuAnchor(
    menuChildren: [
      for (final entry in CrosswordSize.values)
        MenuItemButton(
          onPressed: () => ref.read(sizeProvider.notifier).setSize(entry),
          leadingIcon: entry == ref.watch(sizeProvider)
              ? Icon(Icons.radio_button_checked_outlined)
              : Icon(Icons.radio_button_unchecked_outlined),
          child: Text(entry.label),
        ),
      MenuItemButton(
        leadingIcon: ref.watch(showDisplayInfoProvider)
            ? Icon(Icons.check_box_outlined)
            : Icon(Icons.check_box_outline_blank_outlined),
        onPressed: () => ref.read(showDisplayInfoProvider.notifier).toggle(),
        child: Text('Display Info'),
      ),
      for (final count in BackgroundWorkers.values)        // Add from here
        MenuItemButton(
          leadingIcon: count == ref.watch(workerCountProvider)
              ? Icon(Icons.radio_button_checked_outlined)
              : Icon(Icons.radio_button_unchecked_outlined),
          onPressed: () =>
              ref.read(workerCountProvider.notifier).setCount(count),
          child: Text(count.label),                        // To here.
        ),
    ],
    builder: (context, controller, child) => IconButton(
      onPressed: () => controller.open(),
      icon: Icon(Icons.settings),
    ),
  );
}
```

如果現在執行應用程式，您就能修改要例項化的背景隔離數量，以便搜尋要填入填字遊戲的字詞。

7. 按一下齒輪圖示，開啟內容選單，當中包含填字遊戲的大小、是否顯示生成的填字遊戲統計資料，以及現在要使用的隔離數。



檢查點：多執行緒效能

使用多個核心並行執行填字遊戲產生器，大幅縮短 80x44 填字遊戲的運算時間。請注意：

- 增加工作人員人數，加快生成填字遊戲的速度
- 生成期間 UI 回應速度流暢
- 顯示生成進度的即時統計資料
- 演算法探索區域的視覺化回饋

步驟 9 · 約 15 分鐘

9. 將其變成遊戲

我們要建構的內容：可玩的填字遊戲

最後一節其實是加分題。您將運用建構填字遊戲產生器時學到的所有技巧，建構遊戲。您將學會以下內容：

1. **產生字謎**：使用字謎產生器建立可解的字謎
2. **建立字詞選項**：為每個位置提供多個字詞選項
3. **啟用互動**：允許使用者選取及放置單字
4. **驗證解答**：檢查填好的填字遊戲是否正確

您將使用填字遊戲產生器建立填字遊戲。您將重複使用內容選單慣用語，讓使用者選取及取消選取字詞，填入格線中各種形狀的空格。目標是完成填字遊戲。

我不會說這款遊戲已經完成或打磨完畢，事實上還差得遠。如果替代字詞的選擇不當，可能會導致平衡和難度問題。沒有引導使用者進入謎題的教學課程。我甚至不會提到最基本的「你贏了！」畫面。

但缺點是，要將這個原型遊戲打造成完整遊戲，需要編寫的程式碼會大幅增加。程式碼量超出單一程式碼研究室的範圍。因此，這個步驟是速成練習，目的是改變使用方式和位置，藉此加強您在本程式碼研究室中學到的技巧。希望這能加深您對本程式碼研究室先前課程的印象。或者，您也可以根據這段程式碼自行建構體驗。我們很期待看到您的成果！

如要開始，請按照下列步驟操作：

1. 刪除 `lib/widgets` 目錄中的所有內容。您將為遊戲建立全新的小工具。只是借用了許多舊版小工具的元素。

> **程式碼重複使用策略**：雖然我們會刪除舊的小工具，但我們學到的概念和模式 (Riverpod 提供者、不可變動的資料結構、有效率的 UI 更新) 會在遊戲實作中重複使用。

2. 編輯 `model.dart` 檔案，按照以下方式更新 `Crossword` 的 `addWord` 方法：

lib/model.dart

```
/// Add a word to the crossword at the given location and direction.
Crossword? addWord({
  required Location location,
  required String word,
  required Direction direction,
  bool requireOverlap = true,                              // Add this parameter
}) {
  // Require that the word is not already in the crossword.
  if (words.map((crosswordWord) => crosswordWord.word).contains(word)) {
    return null;
  }

  final wordCharacters = word.characters;
  bool overlap = false;

  // Check that the word fits in the crossword.
  for (final (index, character) in wordCharacters.indexed) {
    final characterLocation = switch (direction) {
      Direction.across => location.rightOffset(index),
      Direction.down => location.downOffset(index),
    };

    final target = characters[characterLocation];
    if (target != null) {
      overlap = true;
      if (target.character != character) {
        return null;
      }
      if (direction == Direction.across && target.acrossWord != null ||
          direction == Direction.down && target.downWord != null) {
        return null;
      }
    }
  }
                                                           // Edit from here
  // If overlap is required, make sure that the word overlaps with an existing
  // word. Skip this test if the crossword is empty.
  if (words.isNotEmpty && !overlap && requireOverlap) {  // To here.
    return null;
  }

  final candidate = rebuild(
    (b) => b
      ..words.add(
        CrosswordWord.word(
          word: word,
          direction: direction,
          location: location,
        ),
      ),
  );
```

```
    if (candidate.valid) {
      return candidate;
    } else {
      return null;
    }
  }
```

只要稍微修改填字遊戲模型，就能加入不重疊的字詞。允許玩家在棋盤上的任何位置下棋，並仍可使用 `Crossword` 做為儲存玩家步數的基本模型，這非常實用。這只是一份清單，列出特定位置的字詞，並以特定方向排列。

3. 在 `model.dart` 檔案結尾新增 `CrosswordPuzzleGame` 模型類別。

lib/model.dart

```
/// Creates a puzzle from a crossword and a set of candidate words.
abstract class CrosswordPuzzleGame
    implements Built<CrosswordPuzzleGame, CrosswordPuzzleGameBuilder> {
  static Serializer<CrosswordPuzzleGame> get serializer =>
      _$crosswordPuzzleGameSerializer;

  /// The [Crossword] that this puzzle is based on.
  Crossword get crossword;

  /// The alternate words for each [CrosswordWord] in the crossword.
  BuiltMap<Location, BuiltMap<Direction, BuiltList<String>>> get alternateWords;

  /// The player's selected words.
  BuiltList<CrosswordWord> get selectedWords;

  bool canSelectWord({
    required Location location,
    required String word,
    required Direction direction,
  }) {
    final crosswordWord = CrosswordWord.word(
      word: word,
      location: location,
      direction: direction,
    );

    if (selectedWords.contains(crosswordWord)) {
      return true;
    }

    var puzzle = this;

    if (puzzle.selectedWords
        .where((b) => b.direction == direction && b.location == location)
        .isNotEmpty) {
      puzzle = puzzle.rebuild(
        (b) => b
          ..selectedWords.removeWhere(
            (selectedWord) =>
                selectedWord.location == location &&
                selectedWord.direction == direction,
          ),
      );
    }

    return null !=
        puzzle.crosswordFromSelectedWords.addWord(
          location: location,
          word: word,
          direction: direction,
          requireOverlap: false,
        );
```

```
}

CrosswordPuzzleGame? selectWord({
  required Location location,
  required String word,
  required Direction direction,
}) {
  final crosswordWord = CrosswordWord.word(
    word: word,
    location: location,
    direction: direction,
  );

  if (selectedWords.contains(crosswordWord)) {
    return rebuild((b) => b.selectedWords.remove(crosswordWord));
  }

  var puzzle = this;

  if (puzzle.selectedWords
      .where((b) => b.direction == direction && b.location == location)
      .isNotEmpty) {
    puzzle = puzzle.rebuild(
      (b) => b
        ..selectedWords.removeWhere(
          (selectedWord) =>
              selectedWord.location == location &&
              selectedWord.direction == direction,
        ),
    );
  }

  // Check if the selected word meshes with the already selected words.
  // Note this version of the crossword does not enforce overlap to
  // allow the player to select words anywhere on the grid. Enforcing words
  // to be solved in order is a possible alternative.
  final updatedSelectedWordsCrossword = puzzle.crosswordFromSelectedWords
      .addWord(
        location: location,
        word: word,
        direction: direction,
        requireOverlap: false,
      );

  // Make sure the selected word is in the crossword or is an alternate word.
  if (updatedSelectedWordsCrossword != null) {
    if (puzzle.crossword.words.contains(crosswordWord) ||
        puzzle.alternateWords[location]?[direction]?.contains(word) == true) {
      return puzzle.rebuild(
        (b) => b
          ..selectedWords.add(
            CrosswordWord.word(
```

```
              word: word,
              location: location,
              direction: direction,
            ),
          ),
      );
    }
  }
  return null;
}

/// The crossword from the selected words.
Crossword get crosswordFromSelectedWords => Crossword.crossword(
  width: crossword.width,
  height: crossword.height,
  words: selectedWords,
);

/// Test if the puzzle is solved. Note, this allows for the possibility of
/// multiple solutions.
bool get solved =>
    crosswordFromSelectedWords.valid &&
    crosswordFromSelectedWords.words.length == crossword.words.length &&
    crossword.words.isNotEmpty;

/// Create a crossword puzzle game from a crossword and a set of candidate
/// words.
factory CrosswordPuzzleGame.from({
  required Crossword crossword,
  required BuiltSet<String> candidateWords,
}) {
  // Remove all of the currently used words from the list of candidates
  candidateWords = candidateWords.rebuild(
    (p0) => p0.removeAll(crossword.words.map((p1) => p1.word)),
  );

  // This is the list of alternate words for each word in the crossword
  var alternates =
      BuiltMap<Location, BuiltMap<Direction, BuiltList<String>>>();

  // Build the alternate words for each word in the crossword
  for (final crosswordWord in crossword.words) {
    final alternateWords = candidateWords.toBuiltList().rebuild(
      (b) => b
        ..where((b) => b.length == crosswordWord.word.length)
        ..shuffle()
        ..take(4)
        ..sort(),
    );

    candidateWords = candidateWords.rebuild(
      (b) => b.removeAll(alternateWords),
```

```
      );

      alternates = alternates.rebuild(
        (b) => b.updateValue(
          crosswordWord.location,
          (b) => b.rebuild(
            (b) => b.updateValue(
              crosswordWord.direction,
              (b) => b.rebuild((b) => b.replace(alternateWords)),
              ifAbsent: () => alternateWords,
            ),
          ),
          ifAbsent: () => {crosswordWord.direction: alternateWords}.build(),
        ),
      );
    }

    return CrosswordPuzzleGame((b) {
      b
        ..crossword.replace(crossword)
        ..alternateWords.replace(alternates);
    });
  }

  factory CrosswordPuzzleGame([
    void Function(CrosswordPuzzleGameBuilder)? updates,
  ]) = _$CrosswordPuzzleGame;
  CrosswordPuzzleGame._();
}

/// Construct the serialization/deserialization code for the data model.
@SerializersFor([
  Location,
  Crossword,
  CrosswordWord,
  CrosswordCharacter,
  WorkQueue,
  DisplayInfo,
  CrosswordPuzzleGame,                                    // Add this line
])
final Serializers serializers = _$serializers;
```

`providers.dart` 檔案的更新內容相當有趣，大多數用於支援統計資料收集的供應商都已移除。我們已移除變更背景隔離數量的功能，並改用常數。此外，還有新的供應器可存取您先前新增的 `CrosswordPuzzleGame` 模型。

lib/providers.dart

```
import 'dart:convert';
                                                                  // Drop the dart:math import

import 'package:built_collection/built_collection.dart';
import 'package:flutter/foundation.dart';
import 'package:flutter/services.dart';
import 'package:riverpod/riverpod.dart';
import 'package:riverpod_annotation/riverpod_annotation.dart';

import 'isolates.dart';
import 'model.dart' as model;

part 'providers.g.dart';

const backgroundWorkerCount = 4;                              // Add this line

/// A provider for the wordlist to use when generating the crossword.
@riverpod
Future<BuiltSet<String>> wordList(Ref ref) async {
  // This codebase requires that all words consist of lowercase characters
  // in the range 'a'-'z'. Words containing uppercase letters will be
  // lowercased, and words containing runes outside this range will
  // be removed.

  final re = RegExp(r'^[a-z]+$');
  final words = await rootBundle.loadString('assets/words.txt');
  return const LineSplitter()
      .convert(words)
      .toBuiltSet()
      .rebuild(
        (b) => b
          ..map((word) => word.toLowerCase().trim())
          ..where((word) => word.length > 2)
          ..where((word) => re.hasMatch(word)),
      );
}

/// An enumeration for different sizes of [model.Crossword]s.
enum CrosswordSize {
  small(width: 20, height: 11),
  medium(width: 40, height: 22),
  large(width: 80, height: 44),
  xlarge(width: 160, height: 88),
  xxlarge(width: 500, height: 500);

  const CrosswordSize({required this.width, required this.height});

  final int width;
  final int height;
  String get label => '$width x $height';
}
```

```
/// A provider that holds the current size of the crossword to generate.
@Riverpod(keepAlive: true)
class Size extends _$Size {
  var _size = CrosswordSize.medium;

  @override
  CrosswordSize build() => _size;

  void setSize(CrosswordSize size) {
    _size = size;
    ref.invalidateSelf();
  }
}

@riverpod
Stream<model.WorkQueue> workQueue(Ref ref) async* {
  final size = ref.watch(sizeProvider);                    // Drop the
ref.watch(workerCountProvider)
  final wordListAsync = ref.watch(wordListProvider);
  final emptyCrossword = model.Crossword.crossword(
    width: size.width,
    height: size.height,
  );
  final emptyWorkQueue = model.WorkQueue.from(
    crossword: emptyCrossword,
    candidateWords: BuiltSet<String>(),
    startLocation: model.Location.at(0, 0),
  );
                                                           // Drop the startTimeProvider and
endTimeProvider refs
  yield* wordListAsync.when(
    data: (wordList) => exploreCrosswordSolutions(
      crossword: emptyCrossword,
      wordList: wordList,
      maxWorkerCount: backgroundWorkerCount,               // Edit this line
    ),
    error: (error, stackTrace) async* {
      debugPrint('Error loading word list: $error');
      yield emptyWorkQueue;
    },
    loading: () async* {
      yield emptyWorkQueue;
    },
  );
}                                                          // Drop the endTimeProvider ref

@riverpod                                                  // Add from here to end of file
class Puzzle extends _$Puzzle {
  model.CrosswordPuzzleGame _puzzle = model.CrosswordPuzzleGame.from(
    crossword: model.Crossword.crossword(width: 0, height: 0),
    candidateWords: BuiltSet<String>(),
  );
```

```
@override
model.CrosswordPuzzleGame build() {
  final size = ref.watch(sizeProvider);
  final wordList = ref.watch(wordListProvider).value;
  final workQueue = ref.watch(workQueueProvider).value;

  if (wordList != null &&
      workQueue != null &&
      workQueue.isCompleted &&
      (_puzzle.crossword.height != size.height ||
          _puzzle.crossword.width != size.width ||
          _puzzle.crossword != workQueue.crossword)) {
    compute(_puzzleFromCrosswordTrampoline, (
      workQueue.crossword,
      wordList,
    )).then((puzzle) {
      _puzzle = puzzle;
      ref.invalidateSelf();
    });
  }

  return _puzzle;
}

Future<void> selectWord({
  required model.Location location,
  required String word,
  required model.Direction direction,
}) async {
  final candidate = await compute(_puzzleSelectWordTrampoline, (
    _puzzle,
    location,
    word,
    direction,
  ));

  if (candidate != null) {
    _puzzle = candidate;
    ref.invalidateSelf();
  } else {
    debugPrint('Invalid word selection: $word');
  }
}

bool canSelectWord({
  required model.Location location,
  required String word,
  required model.Direction direction,
}) {
  return _puzzle.canSelectWord(
    location: location,
```

```
      word: word,
      direction: direction,
    );
  }
}

// Trampoline functions to disentangle these Isolate target calls from the
// unsendable reference to the [Puzzle] provider.

Future<model.CrosswordPuzzleGame> _puzzleFromCrosswordTrampoline(
  (model.Crossword, BuiltSet<String>) args,
) async =>
    model.CrosswordPuzzleGame.from(crossword: args.$1, candidateWords: args.$2);

model.CrosswordPuzzleGame? _puzzleSelectWordTrampoline(
  (model.CrosswordPuzzleGame, model.Location, String, model.Direction) args,
) => args.$1.selectWord(location: args.$2, word: args.$3, direction: args.$4);
```

`Puzzle` 供應商最有趣的部分是，為了掩蓋從 `Crossword` 和 `wordList` 建立 `CrosswordPuzzleGame` 的費用，以及選取字詞的費用，所採取的策略。如果沒有背景 Isolate 的輔助，執行這兩項動作都會導致 UI 互動緩慢。您可以在背景計算最終結果時，使用一些手法推出中間結果，這樣一來，在背景執行必要計算時，UI 仍能保持回應。

4. 在現在空白的 `lib/widgets` 目錄中，建立包含下列內容的 `crossword_puzzle_app.dart` 檔案：

lib/widgets/crossword_puzzle_app.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../providers.dart';
import 'crossword_generator_widget.dart';
import 'crossword_puzzle_widget.dart';
import 'puzzle_completed_widget.dart';

class CrosswordPuzzleApp extends StatelessWidget {
  const CrosswordPuzzleApp({super.key});

  @override
  Widget build(BuildContext context) {
    return _EagerInitialization(
      child: Scaffold(
        appBar: AppBar(
          actions: [_CrosswordPuzzleAppMenu()],
          titleTextStyle: TextStyle(
            color: Theme.of(context).colorScheme.primary,
            fontSize: 16,
            fontWeight: FontWeight.bold,
          ),
          title: Text('Crossword Puzzle'),
        ),
        body: SafeArea(
          child: Consumer(
            builder: (context, ref, _) {
              final workQueueAsync = ref.watch(workQueueProvider);
              final puzzleSolved = ref.watch(
                puzzleProvider.select((puzzle) => puzzle.solved),
              );

              return workQueueAsync.when(
                data: (workQueue) {
                  if (puzzleSolved) {
                    return PuzzleCompletedWidget();
                  }
                  if (workQueue.isCompleted &&
                      workQueue.crossword.characters.isNotEmpty) {
                    return CrosswordPuzzleWidget();
                  }
                  return CrosswordGeneratorWidget();
                },
                loading: () => Center(child: CircularProgressIndicator()),
                error: (error, stackTrace) => Center(child: Text('$error')),
              );
            },
          ),
        ),
      ),
    );
  }
```

```
}

class _EagerInitialization extends ConsumerWidget {
  const _EagerInitialization({required this.child});
  final Widget child;

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    ref.watch(wordListProvider);
    return child;
  }
}

class _CrosswordPuzzleAppMenu extends ConsumerWidget {
  @override
  Widget build(BuildContext context, WidgetRef ref) => MenuAnchor(
    menuChildren: [
      for (final entry in CrosswordSize.values)
        MenuItemButton(
          onPressed: () => ref.read(sizeProvider.notifier).setSize(entry),
          leadingIcon: entry == ref.watch(sizeProvider)
              ? Icon(Icons.radio_button_checked_outlined)
              : Icon(Icons.radio_button_unchecked_outlined),
          child: Text(entry.label),
        ),
    ],
    builder: (context, controller, child) => IconButton(
      onPressed: () => controller.open(),
      icon: Icon(Icons.settings),
    ),
  );
}
```

您現在應該對這個檔案的大部分內容都相當熟悉。是，會有未定義的小工具，現在就開始修正。

5. 建立 `crossword_generator_widget.dart` 檔案，並加入以下內容：

lib/widgets/crossword_generator_widget.dart

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:two_dimensional_scrollables/two_dimensional_scrollables.dart';

import '../model.dart';
import '../providers.dart';

class CrosswordGeneratorWidget extends ConsumerWidget {
  const CrosswordGeneratorWidget({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final size = ref.watch(sizeProvider);
    return TableView.builder(
      diagonalDragBehavior: DiagonalDragBehavior.free,
      cellBuilder: _buildCell,
      columnCount: size.width,
      columnBuilder: (index) => _buildSpan(context, index),
      rowCount: size.height,
      rowBuilder: (index) => _buildSpan(context, index),
    );
  }

  TableViewCell _buildCell(BuildContext context, TableVicinity vicinity) {
    final location = Location.at(vicinity.column, vicinity.row);

    return TableViewCell(
      child: Consumer(
        builder: (context, ref, _) {
          final character = ref.watch(
            workQueueProvider.select(
              (workQueueAsync) => workQueueAsync.when(
                data: (workQueue) => workQueue.crossword.characters[location],
                error: (error, stackTrace) => null,
                loading: () => null,
              ),
            ),
          );

          final explorationCell = ref.watch(
            workQueueProvider.select(
              (workQueueAsync) => workQueueAsync.when(
                data: (workQueue) =>
                    workQueue.locationsToTry.keys.contains(location),
                error: (error, stackTrace) => false,
                loading: () => false,
              ),
            ),
          );

          if (character != null) {
            return AnimatedContainer(
```

```
              duration: Durations.extralong1,
              curve: Curves.easeInOut,
              color: explorationCell
                  ? Theme.of(context).colorScheme.primary
                  : Theme.of(context).colorScheme.onPrimary,
              child: Center(
                child: AnimatedDefaultTextStyle(
                  duration: Durations.extralong1,
                  curve: Curves.easeInOut,
                  style: TextStyle(
                    fontSize: 24,
                    color: explorationCell
                        ? Theme.of(context).colorScheme.onPrimary
                        : Theme.of(context).colorScheme.primary,
                  ),
                  child: Text('•'), // https://www.compart.com/en/unicode/U+2022
                ),
              ),
            );
          }

          return ColoredBox(
            color: Theme.of(context).colorScheme.primaryContainer,
          );
        },
      ),
    );
  }

  TableSpan _buildSpan(BuildContext context, int index) {
    return TableSpan(
      extent: FixedTableSpanExtent(32),
      foregroundDecoration: TableSpanDecoration(
        border: TableSpanBorder(
          leading: BorderSide(
            color: Theme.of(context).colorScheme.onPrimaryContainer,
          ),
          trailing: BorderSide(
            color: Theme.of(context).colorScheme.onPrimaryContainer,
          ),
        ),
      ),
    );
  }
}
```

這應該也相當熟悉。主要差異在於，現在系統會顯示 Unicode 字元，表示有不明字元，而不是顯示所產生字詞的字元。這方面真的需要改善美觀程度。

6. 建立 `crossword_puzzle_widget.dart` 檔案，並加入以下內容：

lib/widgets/crossword_puzzle_widget.dart

```
import 'package:built_collection/built_collection.dart';
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:two_dimensional_scrollables/two_dimensional_scrollables.dart';

import '../model.dart';
import '../providers.dart';

class CrosswordPuzzleWidget extends ConsumerWidget {
  const CrosswordPuzzleWidget({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final size = ref.watch(sizeProvider);
    return TableView.builder(
      diagonalDragBehavior: DiagonalDragBehavior.free,
      cellBuilder: _buildCell,
      columnCount: size.width,
      columnBuilder: (index) => _buildSpan(context, index),
      rowCount: size.height,
      rowBuilder: (index) => _buildSpan(context, index),
    );
  }

  TableViewCell _buildCell(BuildContext context, TableVicinity vicinity) {
    final location = Location.at(vicinity.column, vicinity.row);

    return TableViewCell(
      child: Consumer(
        builder: (context, ref, _) {
          final character = ref.watch(
            puzzleProvider.select(
              (puzzle) => puzzle.crossword.characters[location],
            ),
          );
          final selectedCharacter = ref.watch(
            puzzleProvider.select(
              (puzzle) =>
                  puzzle.crosswordFromSelectedWords.characters[location],
            ),
          );
          final alternateWords = ref.watch(
            puzzleProvider.select((puzzle) => puzzle.alternateWords),
          );

          if (character != null) {
            final acrossWord = character.acrossWord;
            var acrossWords = BuiltList<String>();
            if (acrossWord != null) {
              acrossWords = acrossWords.rebuild(
                (b) => b
                  ..add(acrossWord.word)
```

```
        ..addAll(
          alternateWords[acrossWord.location]?[acrossWord
                  .direction] ??
              [],
        )
        ..sort(),
    );
  }

  final downWord = character.downWord;
  var downWords = BuiltList<String>();
  if (downWord != null) {
    downWords = downWords.rebuild(
      (b) => b
        ..add(downWord.word)
        ..addAll(
          alternateWords[downWord.location]?[downWord.direction] ??
              [],
        )
        ..sort(),
    );
  }

  return MenuAnchor(
    builder: (context, controller, _) {
      return GestureDetector(
        onTapDown: (details) =>
            controller.open(position: details.localPosition),
        child: AnimatedContainer(
          duration: Durations.extralong1,
          curve: Curves.easeInOut,
          color: Theme.of(context).colorScheme.onPrimary,
          child: Center(
            child: AnimatedDefaultTextStyle(
              duration: Durations.extralong1,
              curve: Curves.easeInOut,
              style: TextStyle(
                fontSize: 24,
                color: Theme.of(context).colorScheme.primary,
              ),
              child: Text(selectedCharacter?.character ?? ''),
            ),
          ),
        ),
      );
    },
    menuChildren: [
      if (acrossWords.isNotEmpty && downWords.isNotEmpty)
        Padding(
          padding: const EdgeInsets.all(4),
          child: Text('Across'),
        ),
```

```
                for (final word in acrossWords)
                  _WordSelectMenuItem(
                    location: acrossWord!.location,
                    word: word,
                    selectedCharacter: selectedCharacter,
                    direction: Direction.across,
                  ),
                if (acrossWords.isNotEmpty && downWords.isNotEmpty)
                  Padding(
                    padding: const EdgeInsets.all(4),
                    child: Text('Down'),
                  ),
                for (final word in downWords)
                  _WordSelectMenuItem(
                    location: downWord!.location,
                    word: word,
                    selectedCharacter: selectedCharacter,
                    direction: Direction.down,
                  ),
              ],
            );
          }

          return ColoredBox(
            color: Theme.of(context).colorScheme.primaryContainer,
          );
        },
      ),
    );
  }

  TableSpan _buildSpan(BuildContext context, int index) {
    return TableSpan(
      extent: FixedTableSpanExtent(32),
      foregroundDecoration: TableSpanDecoration(
        border: TableSpanBorder(
          leading: BorderSide(
            color: Theme.of(context).colorScheme.onPrimaryContainer,
          ),
          trailing: BorderSide(
            color: Theme.of(context).colorScheme.onPrimaryContainer,
          ),
        ),
      ),
    );
  }
}

class _WordSelectMenuItem extends ConsumerWidget {
  const _WordSelectMenuItem({
    required this.location,
    required this.word,
```

```
    required this.selectedCharacter,
    required this.direction,
  });

  final Location location;
  final String word;
  final CrosswordCharacter? selectedCharacter;
  final Direction direction;

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final notifier = ref.read(puzzleProvider.notifier);
    return MenuItemButton(
      onPressed:
          ref.watch(
            puzzleProvider.select(
              (puzzle) => puzzle.canSelectWord(
                location: location,
                word: word,
                direction: direction,
              ),
            ),
          )
          ? () => notifier.selectWord(
              location: location,
              word: word,
              direction: direction,
            )
          : null,
      leadingIcon:
          switch (direction) {
            Direction.across => selectedCharacter?.acrossWord?.word == word,
            Direction.down => selectedCharacter?.downWord?.word == word,
          }
          ? Icon(Icons.radio_button_checked_outlined)
          : Icon(Icons.radio_button_unchecked_outlined),
      child: Text(word),
    );
  }
}
```

這個小工具比上一個小工具更密集，但它是由您過去在其他地方看過的片段所建構而成。現在，使用者點選每個填入的儲存格時，系統會顯示內容選單，列出可選取的字詞。如果已選取字詞，則無法選取衝突的字詞。如要取消選取字詞，使用者可以輕觸該字詞的選單項目。

假設玩家可以選取字詞來填滿整個填字遊戲，您需要「你贏了！」畫面。

7. 建立 `puzzle_completed_widget.dart` 檔案，然後在其中加入下列內容：

lib/widgets/puzzle_completed_widget.dart

```
import 'package:flutter/material.dart';

class PuzzleCompletedWidget extends StatelessWidget {
  const PuzzleCompletedWidget({super.key});

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Text(
        'Puzzle Completed!',
        style: TextStyle(fontSize: 36, fontWeight: FontWeight.bold),
      ),
    );
  }
}
```

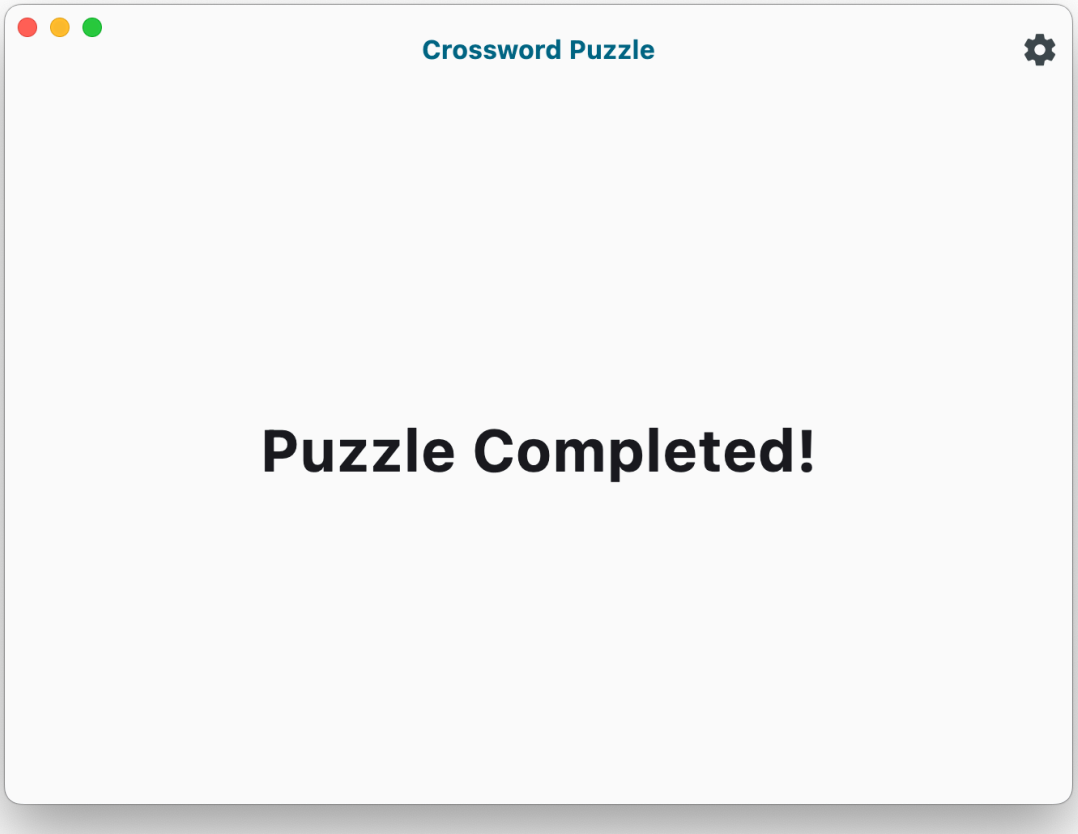
相信你一定能讓這項產品更有趣。如要進一步瞭解動畫工具，請參閱「在 Flutter 中建構新一代 UI」程式碼研究室。

8. 按照下列方式編輯 `lib/main.dart` 檔案：

lib/main.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import 'widgets/crossword_puzzle_app.dart';                // Update this line

void main() {
  runApp(
    ProviderScope(
      child: MaterialApp(
        title: 'Crossword Puzzle',                         // Update this line
        debugShowCheckedModeBanner: false,
        theme: ThemeData(
          colorSchemeSeed: Colors.blueGrey,
          brightness: Brightness.light,
        ),
        home: CrosswordPuzzleApp(),                        // Update this line
      ),
    ),
  );
}
```

執行這個應用程式時，您會看到動畫，因為填字遊戲產生器正在產生謎題。接著，系統會顯示空白的謎題供您解答。假設您已解決問題，畫面上應會顯示類似下方的訊息：


Crossword Puzzle
Puzzle Completed!

步驟 10 · 約 1 分鐘

10. 恭喜

恭喜！您已成功使用 Flutter 建構益智遊戲！

你建構的填字遊戲產生器已成為益智遊戲。您已學會如何在隔離區集區中執行背景運算。您使用了不可變動的資料結構，簡化回溯演算法的實作程序。您也花時間深入瞭解 `TableView`，下次需要顯示表格資料時，就能派上用場。

瞭解詳情

- 深入瞭解 Dart 的模式和記錄
- 在 Flutter 中建構新一代 UI
- 讓 Flutter 應用程式從無趣變為美觀
- Flutter 休閒遊戲工具包
